Cohortdata- DB [OVERVIEW → EVERYTHING] - sameer

Status: Draft v1 — Core database design

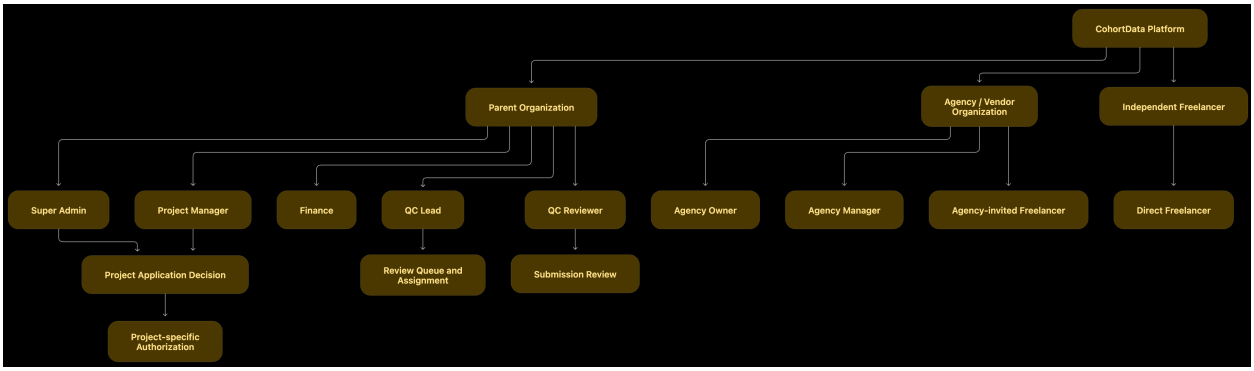
Purpose: Define how users, organizations, permissions, projects, assignments and applications relate to one another before implementation.

1. Database architecture overview

Technology we will be using.

- **PostgreSQL via Supabase:** Main business database and source of truth.
- **Clerk:** Authentication, login, signup and identity verification.
- **Supabase RLS:** Database-level protection for tenant and organization data.
- **Next.js backend:** Enforces business rules and permission checks before performing protected operations.

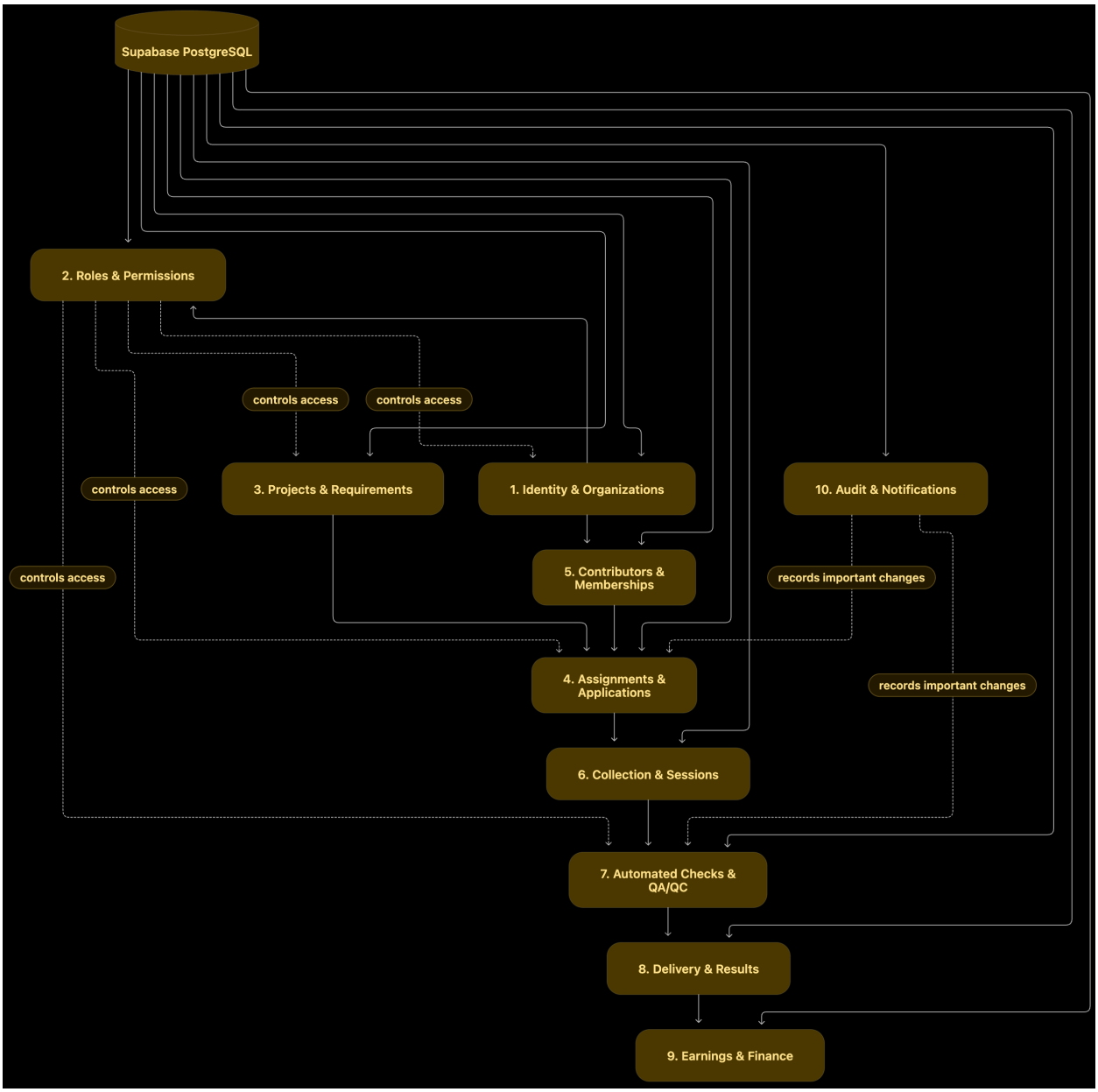
Clerk authenticates the person. CohortData's database determines what that person can access and do.



CohortData — Database Architecture

1. High-level database architecture :

This diagram shows the major parts of the database and how they connect. It is a conceptual overview, not the final table schema.

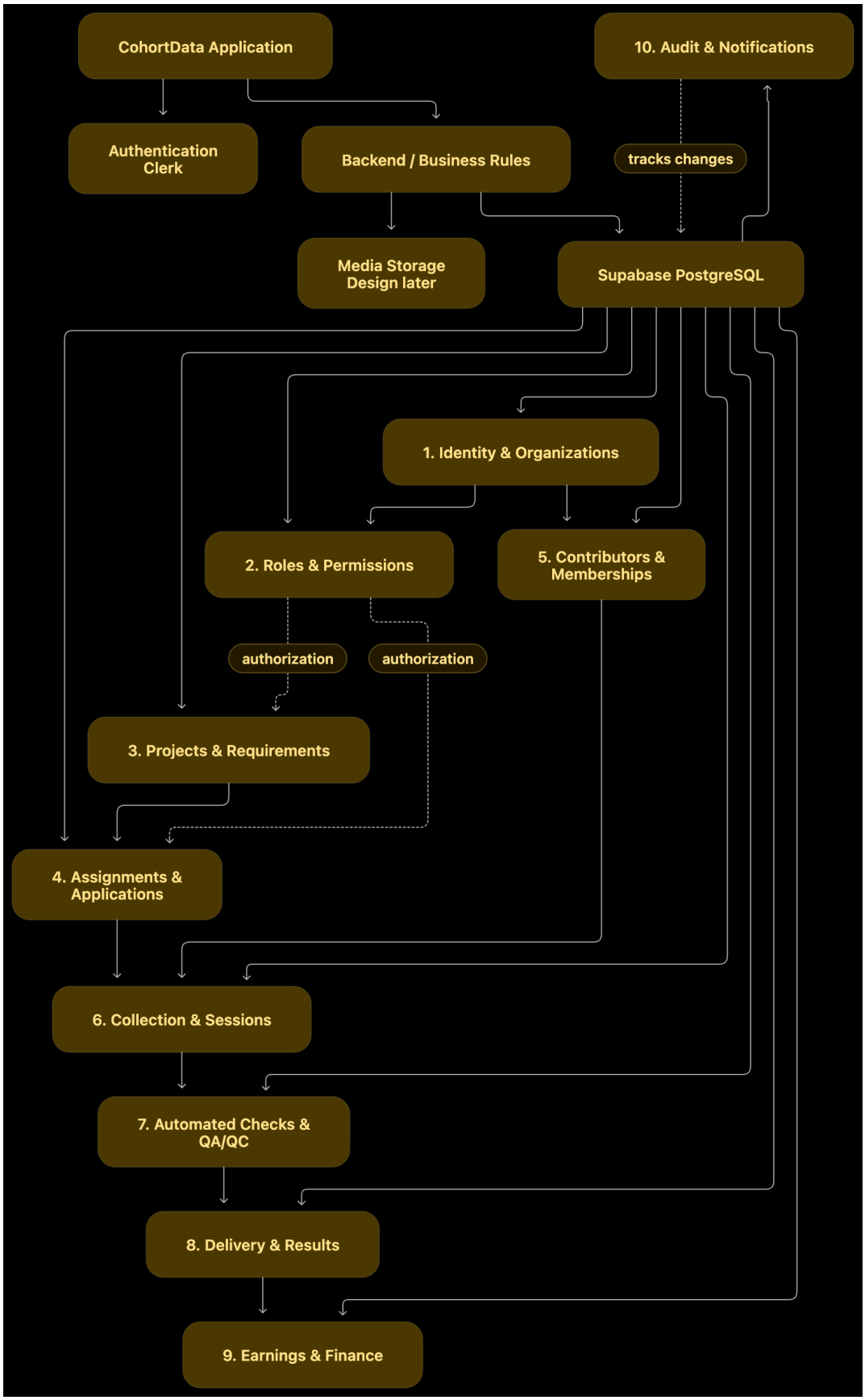


How to read it: Identity establishes who is using the platform. Permissions determine what they can access. Projects define the work, assignments determine who is responsible for it, and the downstream domains track collection through finance.

We will design domains 6–9 in detail later after we complete this. [6-9] considered as Data collection phase

Database architecture — overview level

The database should represent the actual business structure of CohortData, not just the screens in the application.


CohortData Application
10. Audit & Notifications
Authentication Clerk
Backend / Business Rules
tracks changes
Media Storage Design later
Supabase PostgreSQL
1. Identity & Organizations
2. Roles & Permissions
5. Contributors & Memberships
authorization
authorization
3. Projects & Requirements
4. Assignments & Applications
6. Collection & Sessions
7. Automated Checks & QA/QC
8. Delivery & Results
9. Earnings & Finance

2. The database domains we need to design/Architect ：

Domain 1 — Identity & Organizations

Who exists in CohortData?Users, organizations, agencies, organization memberships and contributor identities.

Domain 2 — Roles & Permissions

What is each person allowed to do?Roles, individual permissions, role-to-permission mappings, organization scope and authorization rules.

Domain 3 — Projects & Requirements

What work needs to be done?: Projects, lifecycle status, requirements, eligibility, quotas and project configuration.

Domain 4 — Assignments & Applications

Who gets the work?: Agency assignments, direct freelancer applications, approval decisions and project participation.

Domain 5 — Contributors & Memberships

Who belongs to which organization and performs work? :Agency members, direct freelancers, account-linked profiles and invited contributors without accounts.

Domains 6–10 — Operational data

Sessions and submissions; automated checks and QA/QC; delivery; earnings and finance; audit history and notifications.

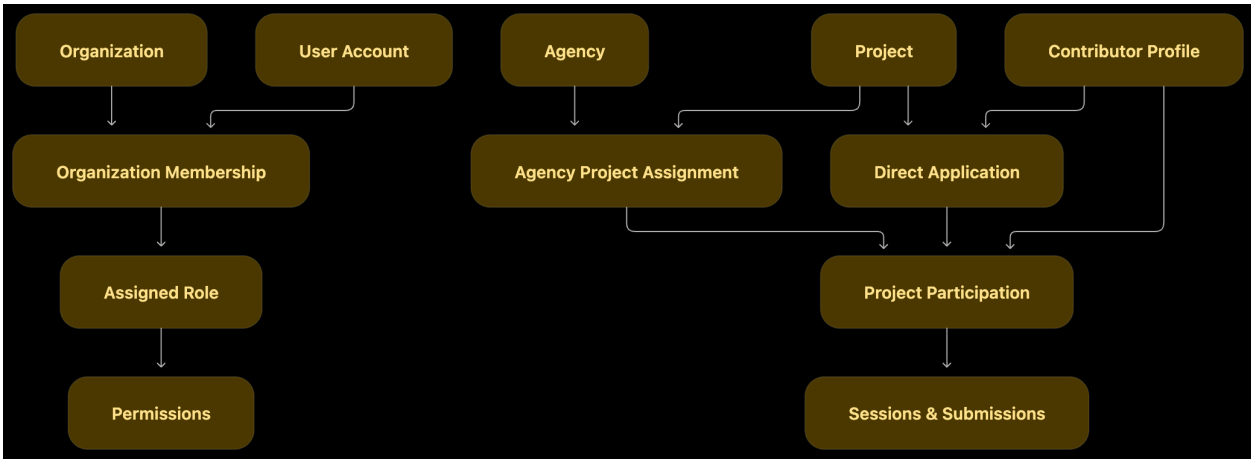
What each database domain is responsible for

Domain	Main question it answers	Examples
1. Identity & Organizations	Who exists in the platform?	Users, agencies, parent organization
2. Roles & Permissions	What can each person do?	Super Admin, Agency Manager, QC Reviewer
3. Projects & Requirements	What work is being requested?	Face Data project, eligibility, targets
4. Assignments & Applications	Who has been offered or approved for the work?	Agency assignments, direct applications
5. Contributors & Memberships	Who belongs to an agency and who performs the work?	Agency members, direct freelancers, invited freelancers
6. Collection & Sessions	How is the work performed and submitted?	Sessions, uploads, metadata
7. Automated Checks & QA/QC	Does the submitted data meet requirements?	Validation, reviews, rejection reasons
8. Delivery & Results	What accepted data has been delivered?	Delivery packages, manifests
9. Earnings & Finance	What money is owed and paid?	Earnings, invoices, payment records
10. Audit & Notifications	What changed, and who needs to know?	Approval history, notifications, audit logs

These are logical domains, not necessarily separate databases. Initially, we can keep them in one PostgreSQL database and organize them through schemas, tables and clear ownership rules.

3. How the main entities relate

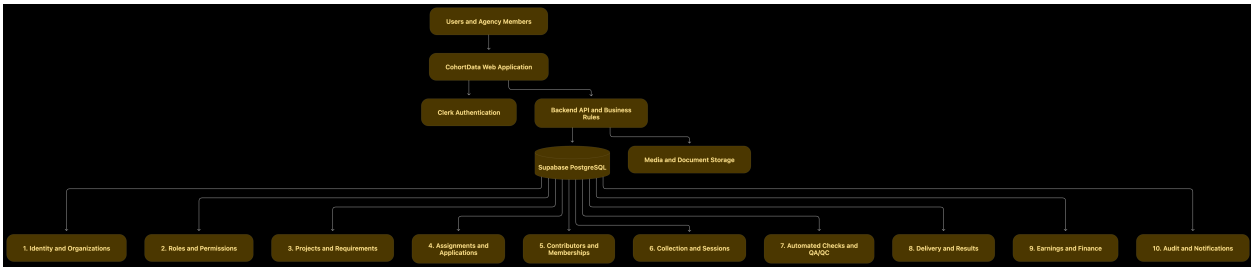
Before looking at individual tables, understand these relationships:



below

1. Is everything here a database concern?

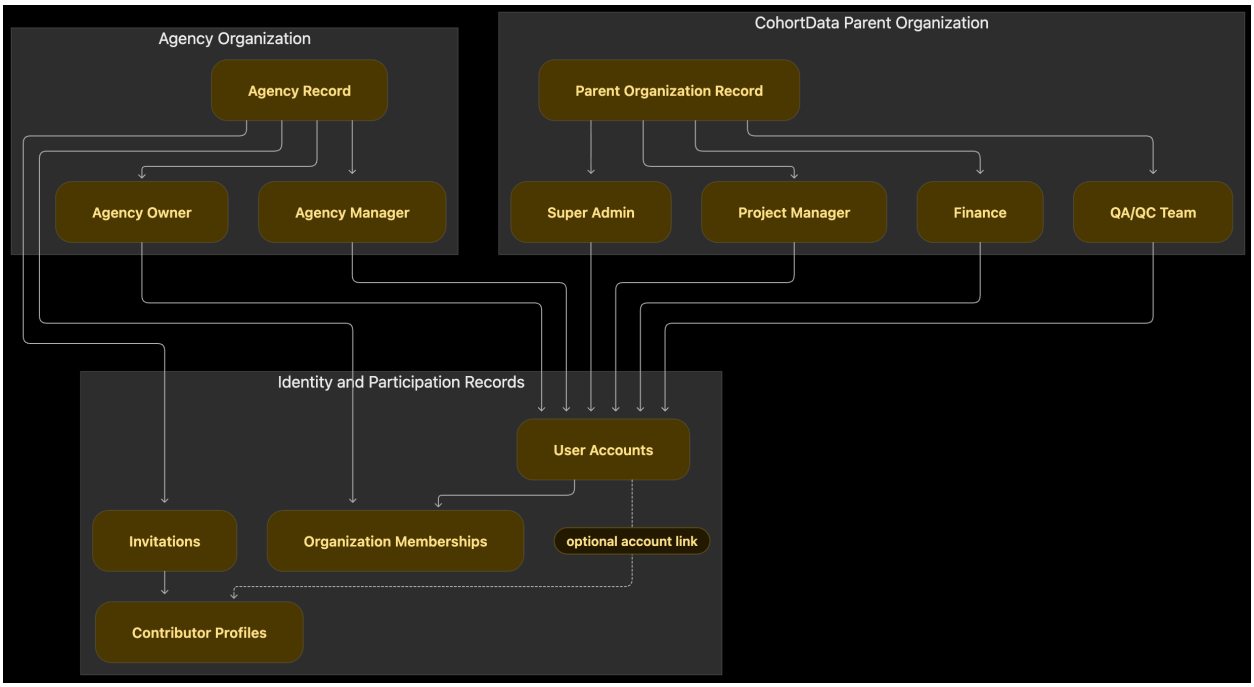
Mostly, yes. The ten domains describe how CohortData's business data is organized. However, authentication, application logic and file storage are separate technical components that interact with the database.



2. Domain 1 — Identity & Organizations

Purpose: Define who exists in CohortData, which organizations exist, and how user accounts and contributor identities relate to those organizations.

2.1 Overview architecture diagram



2.2 Understand the main entities

1. Organization/Agency/Company

Represents CohortData or an agency.

Example: CohortData, Agency A and Agency B are three organization records. Each agency has its own profile and onboarding status.

2. User account

Represents a person who can authenticate through Clerk.

Example: a Super Admin, Agency Owner or Agency Manager. The database stores an internal user ID and a reference to the Clerk account.

3. Organization membership

Connects a user account to an organization.

Example: an Agency Manager is a member of Agency A. Membership records can carry a status such as invited, active or suspended.

what membership - invitiation or active or suspends ?? It means the database tracks each person's relationship with an agency and whether they can access it.

Example: Priya is an Agency Manager at Agency A.

- Invited: Priya has been invited but hasn't joined yet.
- Active: Priya has joined and can access the agency dashboard.
- Suspended: Priya's access has been temporarily disabled.

4. Contributor profile

Represents someone who performs data collection work.

Example: a direct freelancer has an account linked to their contributor profile. An agency-invited freelancer can have a contributor profile without having a login account.

5. Invitation

Tracks an invitation to join an organization or participate in a workflow.

Example: CohortData invites an Agency Owner, or an agency shares a secure collection invitation with a freelancer. These are different invitation purposes and should have clearly defined rules.

2.3 Proposed tables

Table	What it stores
`organizations`	Organization identity, type and status
`agency_profiles`	Agency-specific business and onboarding details
`users`	Internal user record and Clerk account reference
`organization_memberships`	User-to-organization relationships
`contributor_profiles`	People who perform collection work, with optional account links
`invitations`	Invitation purpose, recipient, status, expiry and acceptance

These are proposed names, not yet a finalized schema. We will decide whether agency details belong in `organizations` or `agency_profiles`, and whether different invitation purposes need separate tables, before writing SQL.

2.4 Example data

Organization	Type	Status
CohortData	Parent	Active
Agency A	Agency	Active
Agency B	Agency	Pending approval

User	Organization	Membership
Admin 1	CohortData	Active
Owner 1	Agency A	Active
Manager 1	Agency A	Active

An agency-invited freelancer without an account would have a contributor profile and invitation record, but no user account or login-based organization membership.

2.5 Domain 1 rules

1. Every organization, user and contributor has a stable internal ID.
2. Email is not the primary key.
3. One organization can have many members; a user can potentially belong to multiple organizations.

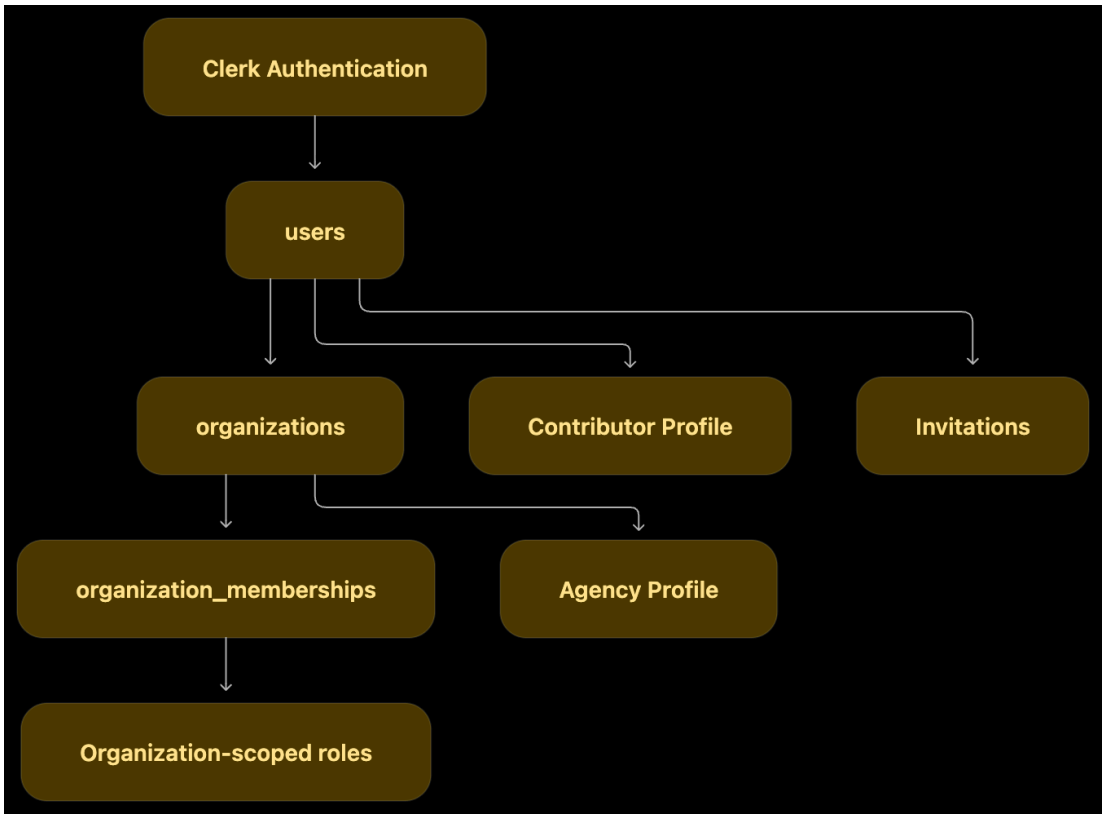
4. Organization membership does not automatically grant project access.
5. Contributor profiles can exist without login accounts.
6. Pending agencies cannot use privileges reserved for approved agencies.
7. Invitations need defined status, expiry and revocation rules.
8. Removing a membership should not erase historical project activity.
9. Authentication and authorization are separate.
10. Personal information and invitation tokens must be protected.

3. Rough principles for the remaining domains

These are initial rules only. Each domain will receive its own architecture diagram, tables, relationships and detailed rules when we work on it.

Domain	Initial design principles
2. Roles & Permissions	Separate roles from permissions; scope access by organization and resource; enforce authorization on the backend and through RLS.
3. Projects & Requirements	Give each project a stable ID; track lifecycle and requirements; preserve important configuration history.
4. Assignments & Applications	Keep agency assignments separate from direct freelancer applications; record decisions, targets and status history.
5. Contributors & Memberships	Separate organizational membership from project participation; support both registered and invited contributors.
6. Collection & Sessions	Track sessions, submissions, consent and metadata; store large files outside PostgreSQL; define expiry and retry rules.
7. Automated Checks & QA/QC	Separate automated results from human reviews; record reviewers, rejection reasons and review history.
8. Delivery & Results	Track accepted data, delivery packages and attempts; distinguish approval from successful delivery.
9. Earnings & Finance	Track eligible earnings, invoice line items, adjustments and manual payments with traceable history.
10. Audit & Notifications	Preserve important actions and status changes; make notifications traceable without treating them as the source of truth.

another thing is



4. What we should finalize first

For Domain 1, the next step is to settle the organization and identity relationships before defining every column.

The key decisions are:

- Is CohortData represented as a normal organization with a special `parent` type?
- Is the agency profile a separate table or part of the organization record?
- Can the same person belong to more than one organization?
- How should an agency owner invitation create the organization and account?
- How do we distinguish an agency's internal member invitation from a no-login contributor invitation?

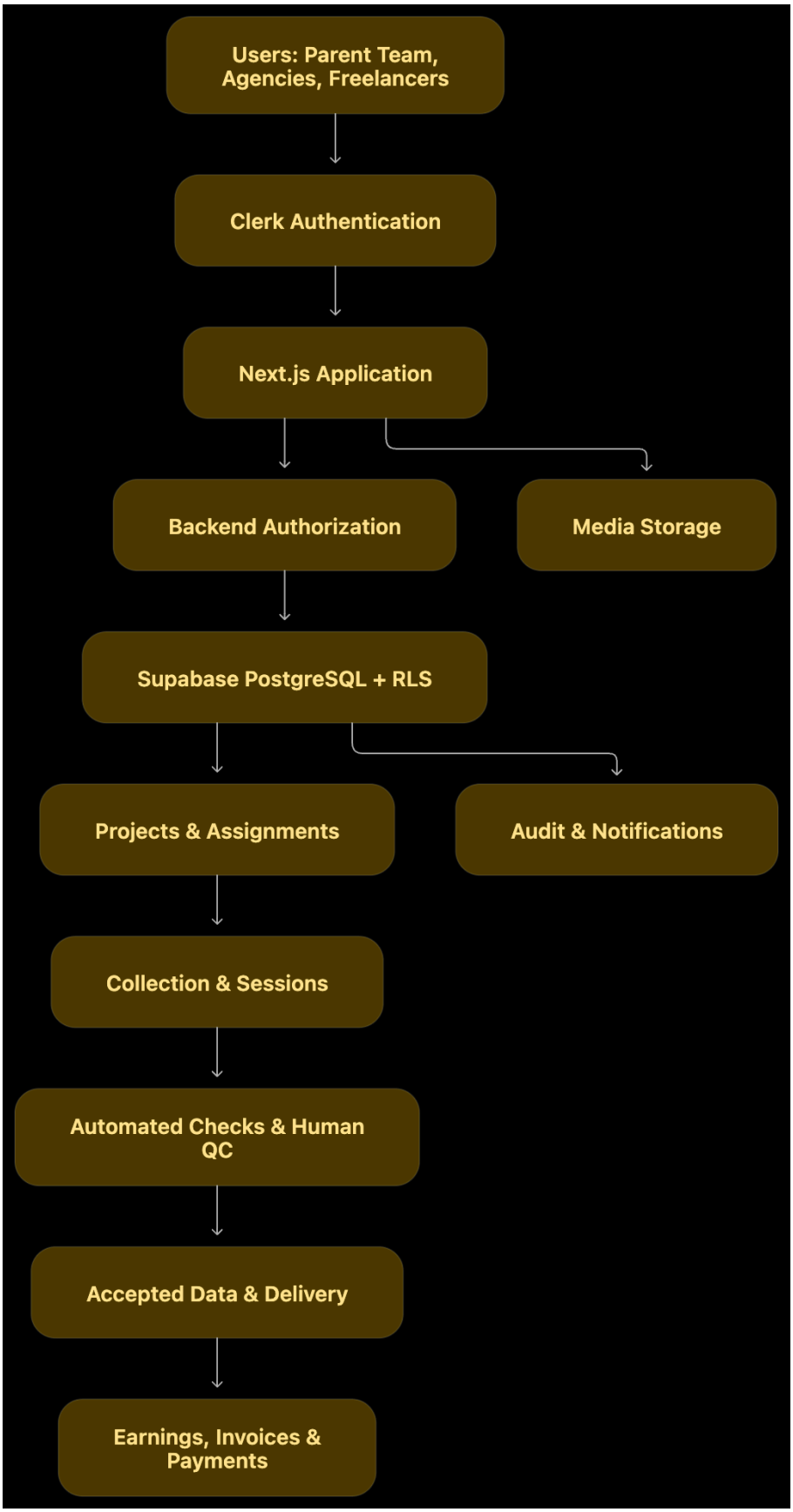
Once these are decided, we can define the exact schema, including primary keys, foreign keys, column types, constraints and example SQL migrations. That becomes the foundation for Domain 2.

lets start this mf thing, one domain by domain full overivew architecture with the supabase terminnology,

CohortData — Complete Database Architecture Blueprint

The goal is to create one clear technical reference for all domains before implementation. It should explain what each domain does, how its workflow operates, which tables it needs, how those tables connect, and which rules the database must enforce.

We'll use the confirmed product requirements and your decisions as the foundation. Anything not yet confirmed will be marked as a proposed design or an open decision, rather than treated as a fact.


Users: Parent Team, Agencies, Freelancers
Clerk Authentication
Next.js Application
Backend Authorization
Media Storage
Supabase PostgreSQL + RLS
Projects & Assignments
Audit & Notifications
Collection & Sessions
Automated Checks & Human QC
Accepted Data & Delivery
Earnings, Invoices & Payments

Architecture terminology

- Clerk: authenticates the person — login, signup, and identity verification.
- PostgreSQL: stores business records and relationships.
- RLS (Row-Level Security): restricts which database records a user can access.
- Backend authorization: checks whether a user is allowed to perform a particular action.
- Object storage: stores large videos, images, audio files, and documents; PostgreSQL stores their metadata and storage references.
- Foreign key (FK): connects a record to another table.
- Primary key (PK): uniquely identifies a record.
- Audit log: records important actions and who performed them.

The key principle is that Clerk proves who the user is; CohortData's database determines what that user can access and do.

2. How the domains fit together

1 Identity & Organizations

Who is the user? Which agency or parent team do they belong to?

2 Roles & Permissions

What actions can they perform, and within which boundaries?

3 Projects & Requirements

What data is needed, under which specifications and quotas?

4 Assignments & Applications

Who applied, who was approved, and who is allowed to work on the project?

5 Contributors & Memberships

Who works directly for CohortData, and who works through an agency?

6 Collection & Sessions

How do people receive links, submit data, and retry rejected submissions?

7 Automated Checks & QA/QC

What passes automated validation, and what needs human review?

8 Delivery & Results

What has been accepted, packaged, delivered, and confirmed?

9 Earnings & Finance

What work qualifies for payment, what is invoiced, and what has actually been paid?

10 Audit & Notifications

What happened, who performed the action, and who needs to be notified?

These are logical domains, not necessarily ten separate applications or databases. We can implement them as modules in one modular monolith using PostgreSQL.

Domain 1 — Identity & Organizations

Purpose

Represent every person, company, and working relationship in CohortData.

A user account, an agency, and a project assignment are three different things. Keeping them separate prevents problems when permissions, agency membership, or project approvals change.

Example flow

A person signs up as an individual freelancer → their account is created → they complete their contributor profile → they apply to an open project → CohortData approves or rejects that application.

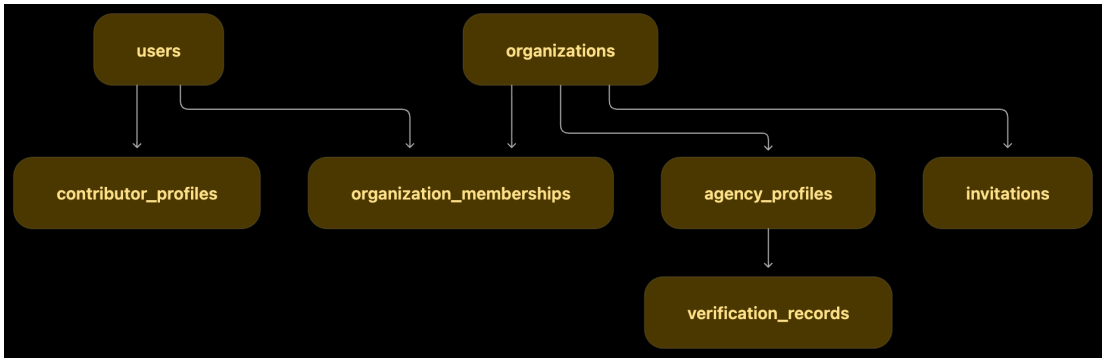
For an agency: the agency owner accepts an invitation or signs up → the agency organization is created or linked → CohortData verifies the agency → the agency applies for projects and invites members.

Proposed tables

Table	Important columns	Purpose
`users`	`id`, `clerk_user_id`, `email`, `status`, `created_at`	Internal representation of a person
`organizations`	`id`, `name`, `type`, `status`, `created_at`	Represents the parent organization or an agency
`organization_memberships`	`id`, `organization_id`, `user_id`, `membership_status`, `joined_at`	Connects a person to an organization
`agency_profiles`	`organization_id`, `verification_status`, `verified_at`, `business_details`	Agency-specific information
`contributor_profiles`	`user_id`, `full_name`, `country`, `profile_status`	Contributor-specific information
`invitations`	`id`, `organization_id`, `email`, `invited_by`, `token_hash`, `expires_at`, `status`	Tracks invitations to join an organization
`verification_records`	`id`, `organization_id`, `reviewed_by`, `status`, `reviewed_at`	Records agency verification decisions

These are proposed table names; the exact columns will be finalized during schema design.

Relationship:



- One organization can have many members.
- One user can have at most one organization membership under the current rule you've chosen.
- An agency organization has one agency profile.
- A user can have a contributor profile independently of organization membership, if the product rules allow it.

Important design decision

The database should distinguish these concepts:

- Authentication identity: Clerk user ID.
- Business identity: internal `users.id`.
- Organization: CohortData or an agency.
- Membership: the user's relationship with an organization.
- Project participation: permission to work on a particular project.

Do not use an agency's name or a user-entered email as the primary key. Use stable IDs.

Open decision: Should direct freelancers have no organization membership, or should each freelancer have an individual organization? The simpler proposed design is no organization membership for direct freelancers.

Domain 2 — Roles & Permissions

Purpose

Determine what each person can do and which records they can access.

A role alone is insufficient. An Agency Owner may manage their agency but must not manage another agency or access CohortData's global finances.

Example flow

A QC Reviewer logs in → the backend identifies their role → the system checks whether the submission is assigned to them → RLS limits database access → the reviewer can record a review only if authorized.

Proposed tables

Table	Important columns	Purpose
`roles`	`id`, `code`, `name`, `scope`	Defines roles such as `PROJECT_MANAGER`
`permissions`	`id`, `code`, `description`	Defines actions such as `submission.review`
`role_permissions`	`role_id`, `permission_id`	Connects roles to allowed actions
`membership_roles`	`membership_id`, `role_id`	Assigns organization-scoped roles
`user_platform_roles`	`user_id`, `role_id`	Assigns CohortData-level roles
`project_access`	`id`, `project_id`, `subject_type`, `subject_id`, `access_status`	Records project-specific authorization where needed

`project_access` is a proposed generic table. It may be unnecessary if project applications and assignments already provide the authoritative access relationship.

Roles

Scope	Role	Responsibility
CohortData	Super Admin	Platform configuration and exceptional overrides
CohortData	Project Manager	Projects, applications, assignments
CohortData	Finance	Platform-wide finance
CohortData	QC Lead	QC queue and reviewer assignments
CohortData	QC Reviewer	Review assigned submissions
Agency	Agency Owner	Agency profile, members, applications, finances
Agency	Agency Manager	Delegated member and operational management
Contributor	Direct Freelancer	Apply for and work on approved projects
Contributor	Agency-invited Freelancer	Submit data through an authorized invitation link

The agency portal is one portal with role-based access, not a separate application for each agency role.

Authorization rules

1. The backend must verify permission for every sensitive action.
2. Supabase RLS must enforce record-level access.
3. An agency can access only its own members, applications, submissions, and finances.
4. A QC Reviewer can review only submissions they are authorized to review.
5. Project approval does not grant platform-wide access.
6. A frontend-provided `organization_id` must never be trusted without server-side validation.

Domain 3 — Projects & Requirements

Purpose

Define what CohortData wants collected, who is eligible to collect it, and how submissions will be evaluated.

A project is the parent record. Its requirements specify what constitutes a valid submission.

Example flow

Project Manager creates a project → configures data type, quotas, instructions, consent, and rates → publishes it → agencies and freelancers can see it according to visibility and eligibility rules. so that they can apply for the project/ again PM and super admin can see the applications and review them and accept or rejects the application.

Proposed tables

Table	Important columns	Purpose
`projects`	`id`, `name`, `description`, `status`, `visibility`, `created_by`, `created_at`	Main project record
`project_versions`	`id`, `project_id`, `version_number`, `configuration`, `created_by`	Preserves important configuration history
`project_requirements`	`id`, `project_id`, `version_id`, `requirement_type`, `configuration`	Defines data and technical requirements
`project_quotas`	`id`, `project_id`, `segment`, `target_count`	Defines collection targets
`project_eligibility_rules`	`id`, `project_id`, `rule_type`, `configuration`	Defines who can participate
`project_consent_templates`	`id`, `project_id`, `version`, `document_reference`	Defines the consent wording applicable to a project
`project_rate_cards`	`id`, `project_id`, `currency`, `rate_configuration`	Defines project compensation rules

JSON configuration can be useful for flexible requirements, but frequently queried attributes should use typed columns or normalized child tables. Avoid putting the entire business model into one JSON field.

Example project

Face Data Collection

Published

Image / Video

Target

10,000 accepted submissions

Requirements

Lighting, background, face visibility

QC

Automated checks + human review

Compensation

Configured rate card

Lifecycle

`DRAFT → PUBLISHED → PAUSED → COMPLETED → ARCHIVED`

Not every transition must be allowed. For example, an archived project should not accept new submissions.

in short explantion:

It's the project lifecycle — the different stages a project goes through from creation to closure.

- DRAFT — Project Manager is creating the project; it isn't visible to contributors yet.
- PUBLISHED — Project is live; eligible agencies and freelancers can apply.
- PAUSED — Temporarily stops new applications or submissions.
- COMPLETED — Collection work is finished.
- ARCHIVED — Project is closed and retained for historical records.

Example: Create a Face Data Collection project → publish it → pause collection if needed → complete the target → archive it.

Configuration history

If the project requirements change, preserve the old version. Each submission should reference the applicable project version so the reviewer can determine which requirements applied at submission time.

Domain 4 — Assignments & Applications

Purpose

Track the difference between applying for a project and being authorized to work on it.

An account registration or agency verification does not automatically approve someone for a project.

Example flow

Agency applies to an open project → Project Manager or Super Admin reviews the application → agency is approved → agency can create authorized collection links and invite eligible members.

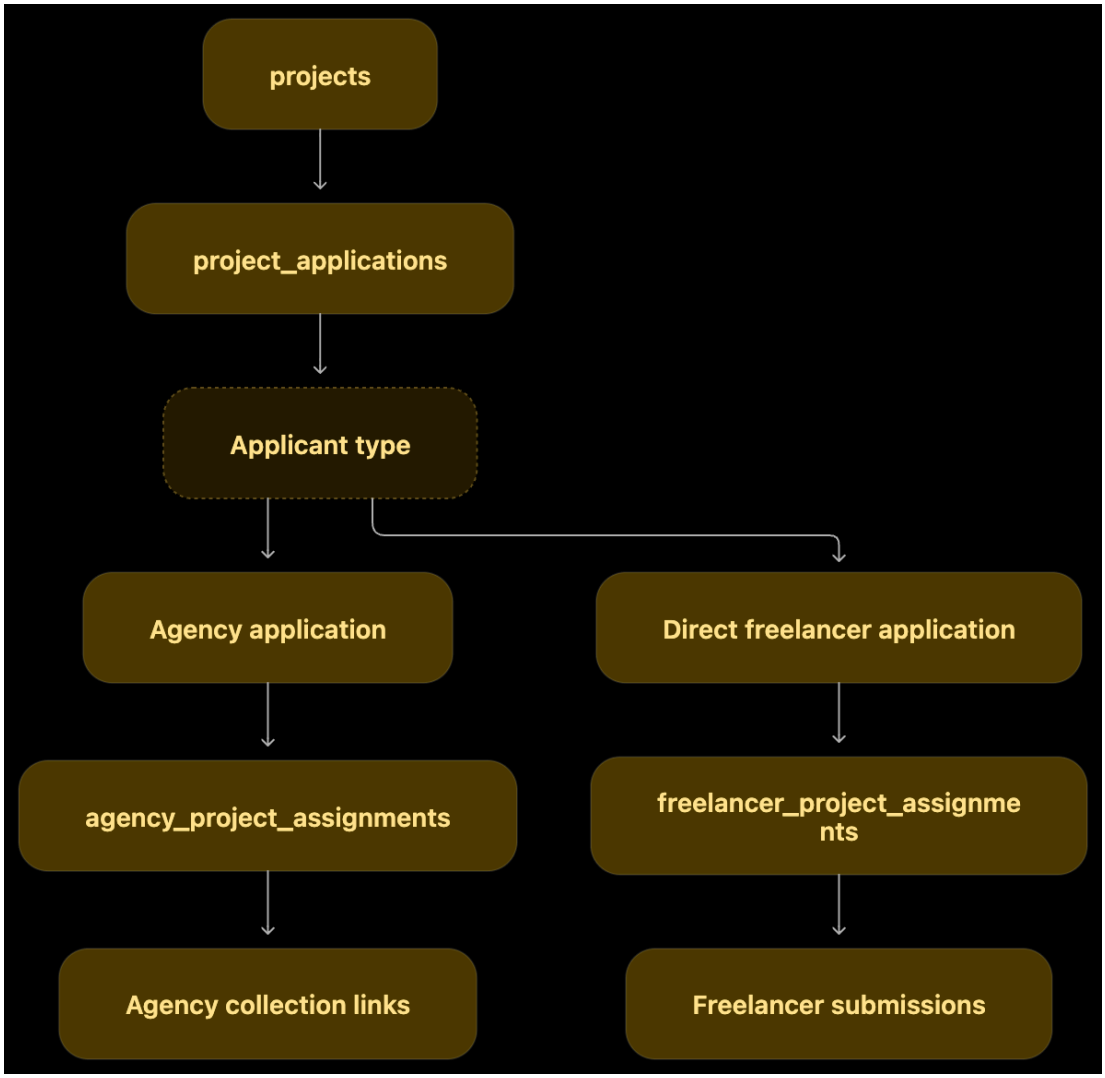
For a direct freelancer, the freelancer applies → CohortData approves that person's application → the freelancer may collect data for that project.

Proposed tables

Table	Important columns	Purpose
`project_applications`	`id`, `project_id`, `applicant_type`, `applicant_id`, `status`, `applied_at`	Records agency or direct freelancer applications
`agency_project_assignments`	`id`, `project_id`, `agency_id`, `status`, `approved_by`, `approved_at`, `target_count`	Records an agency's project authorization
`freelancer_project_assignments`	`id`, `project_id`, `user_id`, `status`, `approved_by`, `approved_at`	Records direct freelancer authorization
`application_decisions`	`id`, `application_id`, `decision`, `decided_by`, `reason`, `created_at`	Preserves approval/rejection decisions
`assignment_status_history`	`id`, `assignment_type`, `assignment_id`, `old_status`, `new_status`, `changed_by`	Tracks changes to assignments

These table boundaries are proposed. We may use a unified assignment table if it remains strongly typed and does not make authorization ambiguous.

Relationships



Key rules

- An agency's verification status and its project approval are separate.
- Only approved agency assignments may create links for that project.
- Only approved direct freelancer assignments may collect data for that project.
- A member working through an agency must be associated with that agency's approved project assignment.
- Store who made every approval or rejection and when.
- Revoking an assignment must prevent new collection activity while preserving existing submissions and audit history.

Important: Project approval is not the same as approval of an individual submission. These are separate lifecycle processes.

Domain 5 — Contributors & Memberships

Purpose

Manage contributors, their profiles, agency membership, and project-specific participation.

A person can be a registered user and still lack permission to participate in a particular project.

Example flow

Agency Owner invites an employee → employee accepts and completes the required profile → the employee is associated with the agency → the employee becomes eligible to submit through that agency's approved project link.

For an invited freelancer who does not log in, the system still needs a way to associate the submission with the correct project and agency without treating a name or email as a verified identity.

Proposed tables

Table	Important columns	Purpose
`contributor_profiles`	`user_id`, `profile_status`, `country`, `qualification_status`	Registered contributor details
`agency_memberships`	`id`, `agency_id`, `user_id`, `status`, `joined_at`	Agency employee/member relationship
`agency_member_invitations`	`id`, `agency_id`, `email`, `invited_by`, `expires_at`, `status`	Invitations to registered agency members
`contributor_qualifications`	`id`, `user_id`, `qualification_type`, `status`, `verified_at`	Tracks qualifications and verification
`contributor_documents`	`id`, `user_id`, `document_type`, `storage_reference`, `verification_status`	Stores document references and verification state

Table	Important columns	Purpose
`project_contributor_participation`	`id`, `project_id`, `contributor_type`, `contributor_id`, `agency_id`, `status`	Optional explicit record of project participation

The final table may be derived from assignments and link usage instead. We should avoid storing the same approval status independently in multiple places.

Two contributor paths

Attribute	Direct freelancer	Agency employee/member	Invited freelancer
Account	Registered	Registered	No account required for collection
Organization	None, under the proposed model	Agency membership	Linked to inviting agency through secure invitation/link context
Project authorization	Direct approval	Agency assignment plus any required member approval	Authorized agency project link
Submission attribution	User ID	User ID and agency ID	Invitation/link ID and agency ID
Profile	Contributor profile	Required contributor profile	Form fields required by collection workflow

The invited freelancer's identity is not automatically verified merely because the form contains a name and email. Duplicate detection and attribution should be separate from identity verification.

Domain 6 — Collection & Sessions

Purpose

Control link creation, collection sessions, submissions, uploaded assets, consent, metadata, and resubmissions.

This domain is especially important because your agency links have different rules for registered employees and non-logged-in freelancers.

Confirmed link rules

- Agencies must be approved for a project before creating its collection links.
- A freelancer link allows collection without login.
- An employee link requires the employee's own account and completed profile.
- A link expires 24 hours after creation.
- A collection session lasts 5 hours after session creation.
- A contributor may submit once per session/link according to the link's applicable rules.
- Rejected submissions may be resubmitted through the same link before its expiry.
- Duplicate submissions may be rejected.
- The agency can track submission progress and outcomes.

One distinction remains important: a 5-hour session and a 24-hour link lifetime are separate clocks.

Example flow

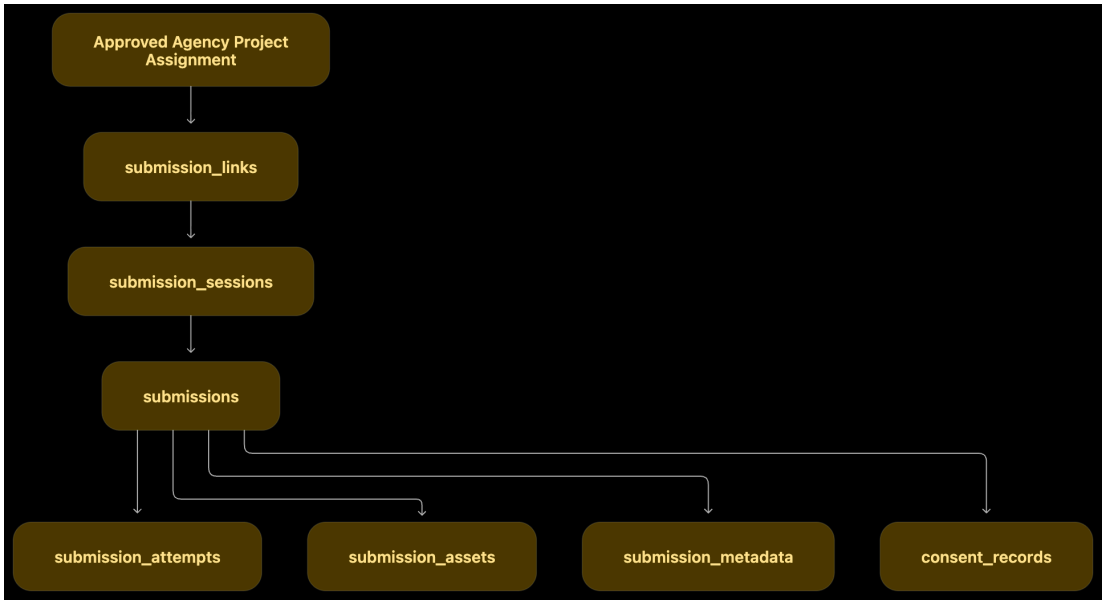
Agency creates a freelancer link → freelancer opens it → a 5-hour session begins → freelancer submits data → automated checks run → if rejected, the link can be reused before expiry → if accepted, the submission enters the QC queue.

Proposed tables

Table	Important columns	Purpose
`submission_links`	`id`, `project_id`, `agency_id`, `assignment_id`, `link_type`, `token_hash`, `created_by`, `expires_at`, `status`	Defines authorized collection links
`submission_sessions`	`id`, `link_id`, `started_at`, `expires_at`, `status`	Tracks individual collection sessions
`submissions`	`id`, `project_id`, `agency_id`, `contributor_user_id`, `link_id`, `session_id`, `status`, `submitted_at`	Main submitted item
`submission_attempts`	`id`, `submission_id`, `attempt_number`, `submitted_at`, `outcome`	Preserves original and resubmission attempts
`submission_assets`	`id`, `submission_id`, `asset_type`, `storage_key`, `checksum`, `file_size`, `mime_type`	References uploaded files
`submission_metadata`	`id`, `submission_id`, `field_key`, `field_value`	Stores collection metadata
`consent_records`	`id`, `submission_id`, `consent_version`, `consented_at`, `evidence_reference`	Records consent associated with the submission

Depending on the final design, metadata may be normalized into project-specific typed fields rather than stored as key/value pairs.

Relationships



Critical implementation rules

1. Generate secure, unguessable link tokens; store token hashes rather than reusable plaintext secrets.
2. Check link expiry, project status, assignment status, and contributor eligibility on the backend.
3. Prevent concurrent duplicate submissions using database constraints or transactional locking.
4. Keep large files in object storage, not PostgreSQL byte columns.
5. Store the file's checksum and storage reference to support integrity checks and duplicate detection.
6. A rejection must not erase the original attempt or its QC decision.
7. Use an explicit rule for session expiry, link expiry, and resubmission eligibility.
8. Never allow a link to authorize submissions for a different project or agency.

Open decision: When a freelancer submits, is the 5-hour session consumed by the first submission regardless of the result, or does a rejected submission reopen the same session? The link may remain valid for 24 hours, but the session behavior needs to be defined separately.

Domain 7 — Automated Checks & QA/QC

Purpose

Keep automated validation separate from human quality review. after the automated qc checks → that goes to the queues and then it goes to the qc lead then he reviews and assign to the qa reviewers.

Automated checks can inspect file type, duration, dimensions, required metadata, technical validity, and duplicate signals. Human reviewers evaluate quality requirements that require judgment.

Example flow

Submission recorded → automated checks run → failed checks produce a rejection or flag according to project rules → passing submissions enter the QC queue → QC Lead assigns a reviewer → reviewer approves or rejects with reasons.

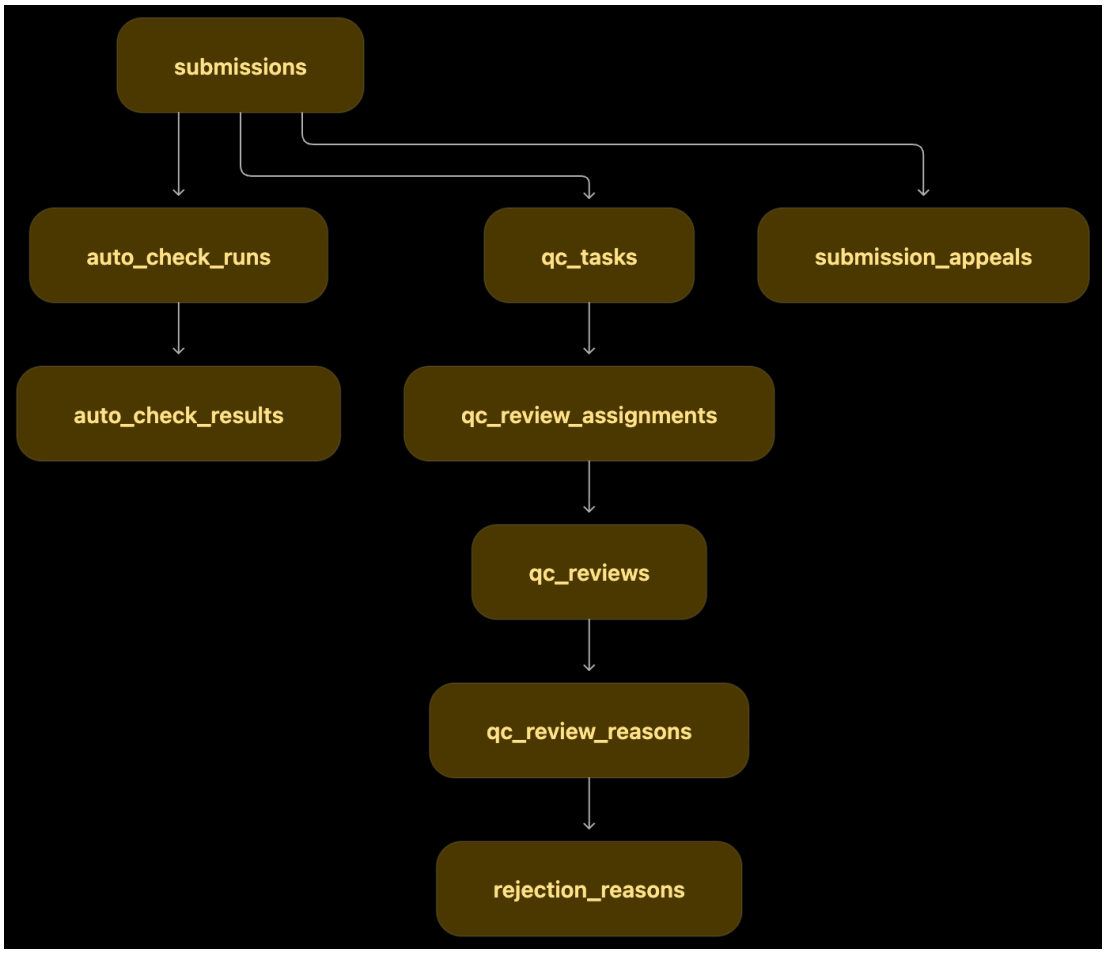
The exact behavior when an automated check fails should be configurable by check type and project. Not every failure must automatically mean final rejection.

Proposed tables

Table	Important columns	Purpose
`auto_check_runs`	`id`, `submission_id`, `status`, `started_at`, `completed_at`, `engine_version`	One execution of automated validation
`auto_check_results`	`id`, `run_id`, `check_code`, `result`, `details`	Result of each individual check
`qc_tasks`	`id`, `submission_id`, `queue_status`, `priority`, `created_at`	Work item for human review
`qc_review_assignments`	`id`, `qc_task_id`, `reviewer_user_id`, `assigned_by`, `assigned_at`, `status`	Reviewer assignment

Table	Important columns	Purpose
`qc_reviews`	`id`, `qc_task_id`, `reviewer_user_id`, `decision`, `comments`, `reviewed_at`	Immutable record of each review
`rejection_reasons`	`id`, `code`, `description`, `category`, `active`	Standardized rejection reasons
`qc_review_reasons`	`review_id`, `reason_id`	Associates a review with one or more reasons
`submission_appeals`	`id`, `submission_id`, `raised_by`, `reason`, `status`, `resolved_by`	Appeal and escalation workflow

Relationship diagram:



Status separation

Use separate status fields for separate processes.

Process	Example statuses
Automated checks	`PENDING`, `RUNNING`, `PASSED`, `FAILED`, `ERROR`
QC task	`QUEUED`, `ASSIGNED`, `IN_REVIEW`, `COMPLETED`

Process	Example statuses
QC decision	`APPROVED`, `REJECTED`, `RESUBMISSION_REQUESTED`
Appeal	`OPEN`, `UNDER_REVIEW`, `UPHELD`, `OVERTURNED`, `CLOSED`

These are proposed enums and can be adjusted to your exact workflow.

Key rules

- Every QC decision records its reviewer, timestamp, decision, and reason.
- The QC Lead manages queues and assignments; the QC Reviewer performs the review.
- Previous reviews remain available when a submission is resubmitted.
- A submission should not enter delivery merely because its automated checks passed.
- Only the authorized role may publish final results if the product requires a separate publication step.

Domain 8 — Delivery & Results

Purpose

Manage accepted data after QC, including packaging, manifests, delivery attempts, and confirmation.

Approved data is not necessarily delivered data. A delivery may fail even after every included submission has been accepted. we need to have the delivered data or failed data everything needs to be logged and shown somewhere in like a seperate portal where we can see

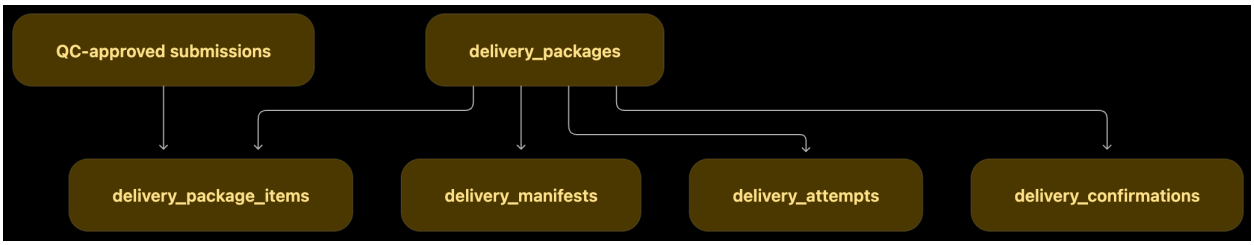
Example flow

QC approves submissions → eligible submissions are selected for a delivery package → files are prepared → manifest is generated → delivery is attempted → successful delivery is recorded.

Proposed tables

Table	Important columns	Purpose
`delivery_packages`	`id`, `project_id`, `status`, `created_by`, `created_at`	Groups accepted submissions for delivery
`delivery_package_items`	`delivery_package_id`, `submission_id`	Connects submissions to packages
`delivery_manifests`	`id`, `package_id`, `version`, `storage_reference`, `checksum`	Records the package contents
`delivery_attempts`	`id`, `package_id`, `attempt_number`, `status`, `started_at`, `completed_at`, `error_details`	Tracks delivery attempts
`delivery_confirmations`	`id`, `package_id`, `confirmed_by`, `confirmed_at`, `confirmation_reference`	Records delivery confirmation

Relationship diagram



Important rules

- A delivery package contains only eligible submissions.
- A submission may need to be excluded if it is disputed, missing consent, or otherwise blocked.
- A package should preserve a record of exactly which submissions it contained.
- Failed delivery attempts must not be overwritten when a retry occurs.
- The system should distinguish `APPROVED`, `PACKAGED`, `DELIVERY_IN_PROGRESS`, `DELIVERED`, and `DELIVERY_FAILED`.
- Whether finance eligibility requires QC approval alone or successful delivery must be explicitly configured.

Domain 9 — Earnings & Finance

Purpose

Track the financial value of accepted work, agency earnings, contributor earnings where applicable, invoices, adjustments, and manual payments.

The PRD specifies manual payment recording initially, rather than automated payouts.

Example flow

Submission becomes financially eligible → system calculates the applicable rate → earning line is recorded → billing period closes → invoice is generated → Finance reviews it → payment is recorded manually.

For agency work, the platform may owe the agency while the agency separately manages its contributors' compensation. Those are different financial relationships and should not be confused.

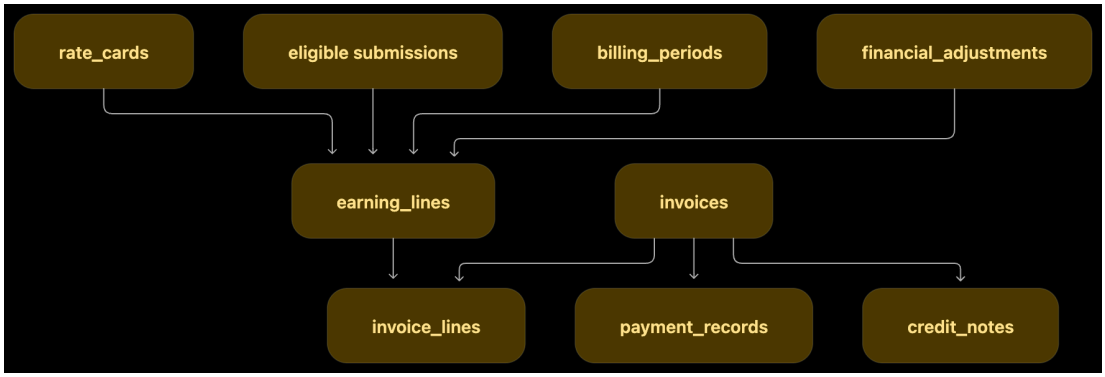
Proposed tables

Table	Important columns	Purpose
`rate_cards`	`id`, `project_id`, `currency`, `effective_from`, `effective_to`, `status`	Defines compensation rules
`rate_card_items`	`id`, `rate_card_id`, `unit_type`, `amount`, `conditions`	Defines rates per item, unit, or condition
`billing_periods`	`id`, `start_date`, `end_date`, `status`, `closed_at`	Defines the accounting period

Table	Important columns	Purpose
`earning_lines`	`id`, `payee_organization_id`, `payee_user_id`, `submission_id`, `amount`, `currency`, `status`	Records eligible earnings
`invoices`	`id`, `payee_organization_id`, `invoice_number`, `currency`, `total_amount`, `status`, `issued_at`	Invoice header
`invoice_lines`	`id`, `invoice_id`, `earning_line_id`, `description`, `quantity`, `unit_rate`, `amount`	Itemized invoice lines
`financial_adjustments`	`id`, `payee_organization_id`, `amount`, `reason`, `approved_by`	Corrections, bonuses, or penalties
`payment_records`	`id`, `invoice_id`, `amount`, `currency`, `payment_reference`, `recorded_by`, `paid_at`	Manual payment records
`credit_notes`	`id`, `invoice_id`, `amount`, `reason`, `status`	Formal invoice corrections where required

A ledger-based accounting design may be appropriate as the financial model matures. For the initial version, we must at least preserve immutable earning and payment history and avoid silently editing historical amounts.

Relationship diagram



Key rules

- Define whether rates apply per submission, accepted item, minute, hour, or another unit.
- Use the rate that was applicable when the earning was created, not today's rate.
- Prevent the same accepted submission from being credited twice for the same earning type.
- Invoice totals should reconcile with invoice lines and adjustments.
- A partial payment must not mark an invoice fully paid.
- Record who created, approved, adjusted, or paid each financial record.
- Agency finance access must be restricted to that agency; parent Finance can access platform-wide records.
- If an agency pays contributors independently, that should not automatically create a CohortData payment record.

Open decision: Is the primary payee always the agency for agency-submitted work, while direct freelancers are paid individually? That decision affects the financial relationships and invoice grouping.

Domain 10 — Audit & Notifications

Purpose

Make important actions traceable and ensure users receive relevant updates without making email or notification records the source of truth.

Example flow

QC Reviewer rejects a submission → database stores the decision and reason → an audit event records who made the decision → notification is queued → Resend delivers the email → delivery status is recorded.

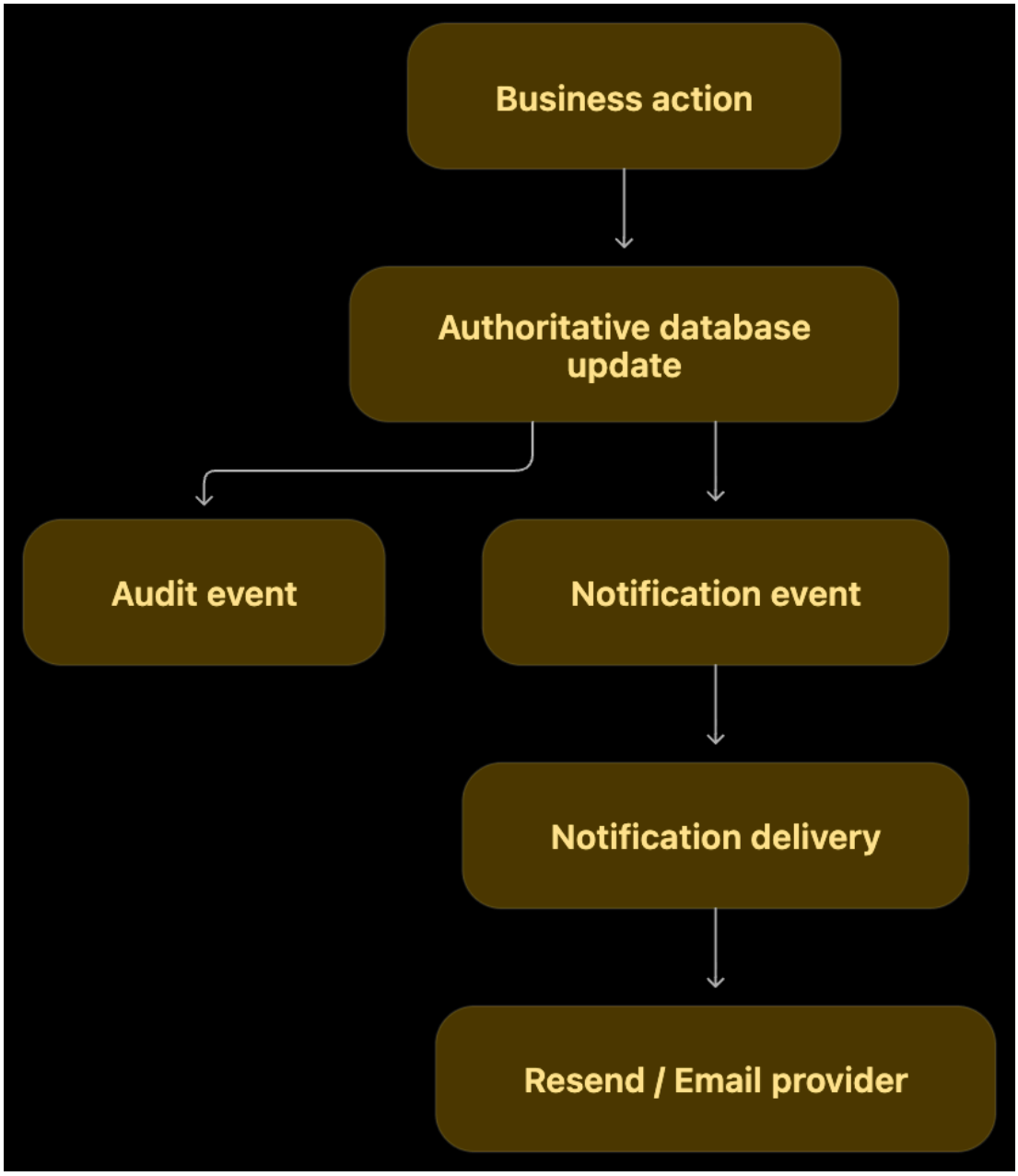
The submission's actual status remains authoritative even if the email fails to send.

Proposed tables

Table	Important columns	Purpose
`audit_logs`	`id`, `actor_user_id`, `action`, `resource_type`, `resource_id`, `created_at`, `metadata`	Records important actions
`notifications`	`id`, `recipient_user_id`, `type`, `resource_type`, `resource_id`, `status`, `created_at`	Tracks notifications to users
`notification_deliveries`	`id`, `notification_id`, `channel`, `provider_reference`, `status`, `attempted_at`	Tracks email or other delivery attempts
`notification_templates`	`id`, `template_key`, `version`, `content`, `active`	Manages message templates
`background_jobs`	`id`, `job_type`, `status`, `attempt_count`, `scheduled_at`, `completed_at`	Optional tracking for asynchronous processing

`background_jobs` may be managed by the chosen job-processing system rather than a custom table. We should add it only if the implementation needs it.

Relationship diagram



Key rules

- Record sensitive actions such as project approval, assignment changes, QC decisions, finance adjustments, and permission changes.
- Audit records should be append-only for ordinary users.
- Store the actor, action, affected resource, timestamp, and relevant context.
- Do not store passwords, raw session tokens, or unnecessary sensitive data in audit metadata.
- Notification retries must not duplicate the underlying business action.
- Failed emails should not reverse a successful database transaction.

Think of it this way: Audits record what happened; notifications tell people what happened.

1. Audit records

An audit record is a permanent history entry that tracks who did what, to which record, and when.

Example: A QC Reviewer rejects a freelancer's video.

The audit record might store:

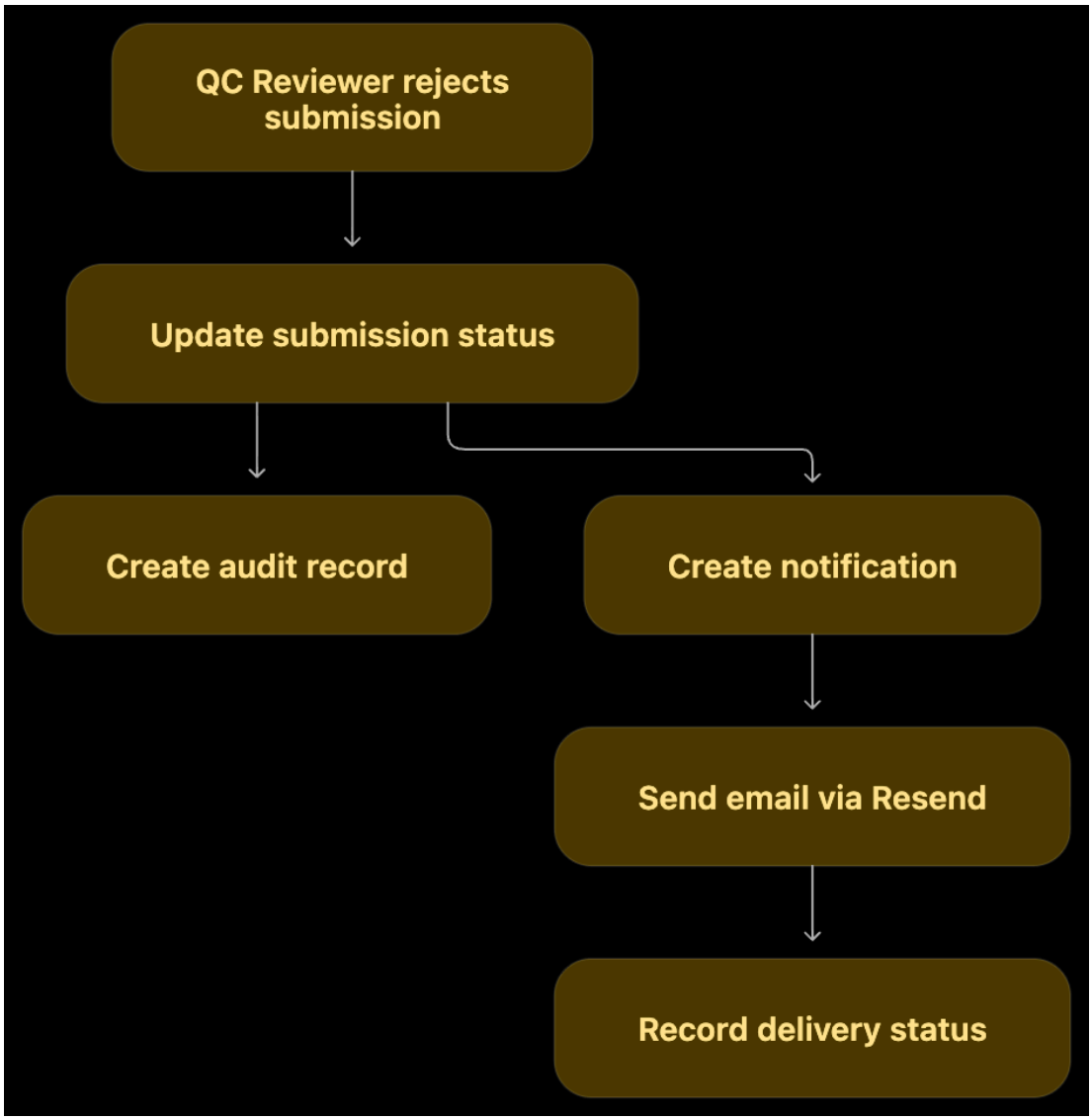
- Who: QC Reviewer — Sameer
- Action: Submission rejected
- Which record: Submission `SUB-102`
- When: October 9, 2026, at 2:30 PM
- Reason: Incorrect lighting

Even if the submission's status later changes, the earlier rejection event remains in the audit history.

2. Notifications

Notifications inform users about important events, through email or in-app messages.

Example: The freelancer's video is rejected. CohortData sends the freelancer an email explaining the rejection and how to resubmit.



What's the difference in the database?

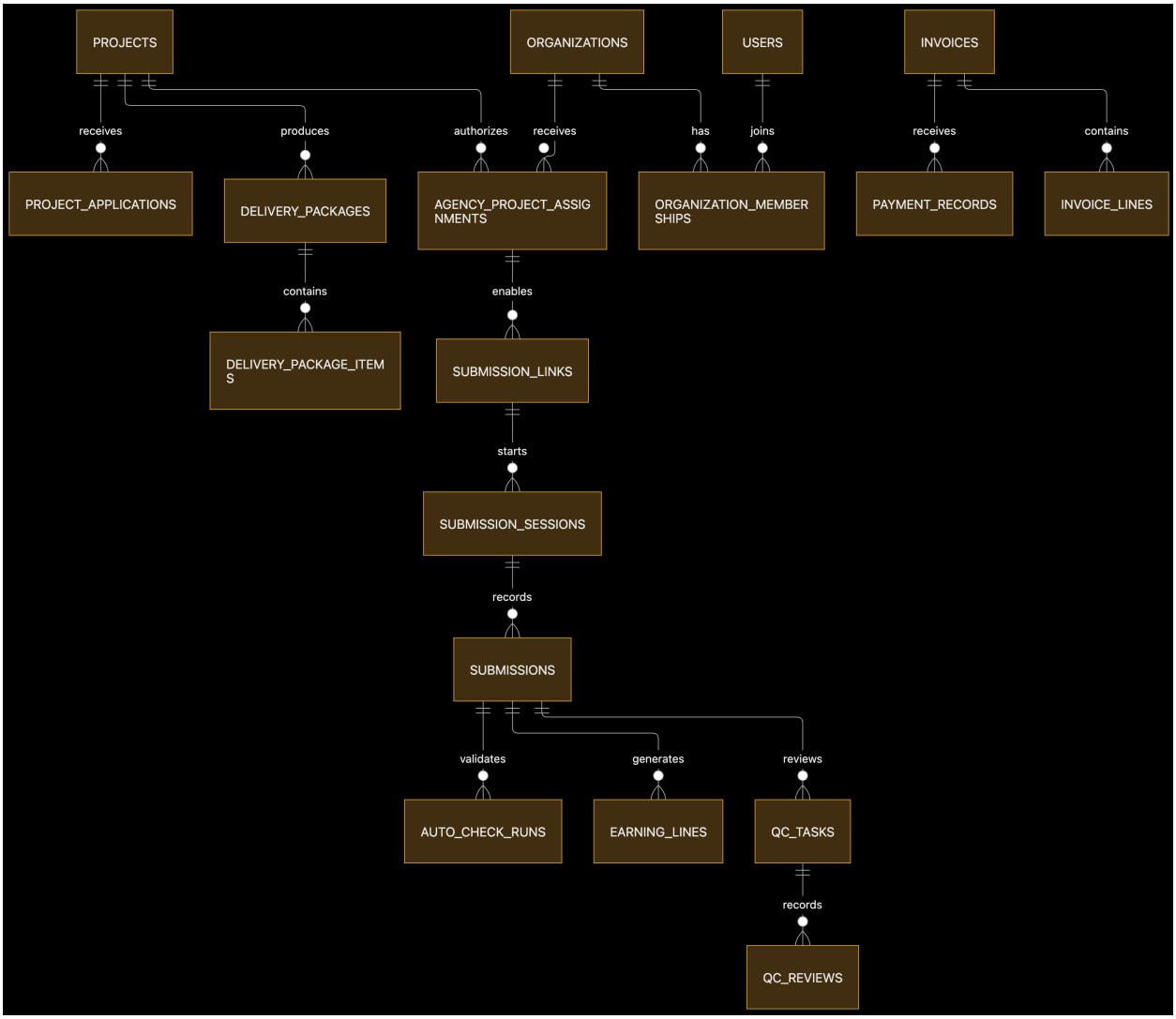
Audit records	Notifications
`audit_logs`	`notifications`
Records actions and changes	Records messages to be sent
Used for accountability and investigations	Used to inform users
Should be protected from modification	Can track pending, sent, or failed delivery
Example: Who rejected a video?	Example: Was the rejection email sent?

In short: If your manager asks, "Who rejected this submission and why?" check the audit log. If they ask, "Did the freelancer receive an email about the rejection?" check notification delivery records.

we need seperate audit logs , where we as a parent company can see the whole audits across all the agencies, but the

3. Cross-domain relationships — the most important database concept

The domains above are not independent. These are the core relationships that connect the whole platform.



This diagram is a conceptual relationship map. It shows the intended connections, not the complete final schema or every required foreign key.

Example: one submission from start to finish

Suppose an agency is approved for a video collection project.

1. The agency receives a project assignment.
2. Its owner creates a freelancer link.
3. A freelancer opens the link and submits a video.
4. The submission is stored against the project, agency, link, and session.
5. Automated checks run and produce results.
6. If accepted by automated checks, the submission enters the QC queue.
7. The QC Lead assigns a QC Reviewer.
8. The reviewer approves the submission.
9. The submission becomes eligible for delivery and, subject to finance rules, earnings.
10. Finance groups eligible earnings into an invoice and records payments.
11. Audit records preserve the important actions throughout the workflow.

The critical point is that one submission record connects the collection, QC, delivery, and finance processes, while each process maintains its own history and status.

4. Data ownership and integrity rules

These rules should apply across all domains.

Data	Owner / scope	Example restriction
Projects	CohortData	Agencies cannot edit global project requirements
Agency profile	Specific agency	Other agencies cannot access its private details
Agency membership	Specific agency	An agency cannot manage another agency's members
Project application	Applicant + project	Applicant sees their application; authorized parent staff can decide it
Submission	Project + contributor/link + agency context	Agency sees only submissions associated with its authorized work
QC review	Submission + authorized reviewer	Review history cannot be edited by unrelated users
Delivery package	Project and delivery workflow	Only eligible submissions can be packaged
Earning and invoice	Payee and finance scope	Agencies see their own financial records
Audit record	Platform-wide audit scope	Ordinary users cannot rewrite audit history

Integrity rules to enforce

- Foreign keys prevent records from referencing nonexistent projects, users, or assignments.
- Unique constraints prevent duplicate memberships, duplicate invoice numbers within their required scope, and duplicate earning entries where applicable.
- Transactions keep related operations consistent.
- Status transitions must be validated by the backend.

- RLS enforces row access; backend authorization enforces business permissions.
- Historical decisions and financial events should be preserved rather than silently overwritten.

5. Database conventions

Use consistent conventions throughout the schema.

Convention	Proposed standard
Primary keys	UUID
Timestamps	`created_at`, `updated_at`, with UTC storage
Foreign keys	Explicit references such as `project_id`
Status fields	Controlled enums or constrained text values
Money	Fixed-precision numeric amounts plus currency code
Files	Object-storage key, checksum, size, MIME type
Soft deletion	Archive or status transition for important business records
History	Separate history/event records for important changes
Secrets	Store hashes of reusable link tokens, not raw tokens

Indexing examples

Indexes should follow real queries, not be added indiscriminately.

- `project_applications(project_id, status)` — list pending applications for a project.
- `submissions(project_id, status, submitted_at)` — power project dashboards and filters.
- `submissions(agency_id, status, submitted_at)` — show agency submission progress.
- `qc_tasks(queue_status, created_at)` — load the QC queue.
- `qc_review_assignments(reviewer_user_id, status)` — show a reviewer's assigned work.
- `invoices(payee_organization_id, status)` — show agency invoices.
- `audit_logs(resource_type, resource_id, created_at)` — inspect an item's history.

These are starting examples. Final indexes depend on actual column names, data volume, and query patterns.

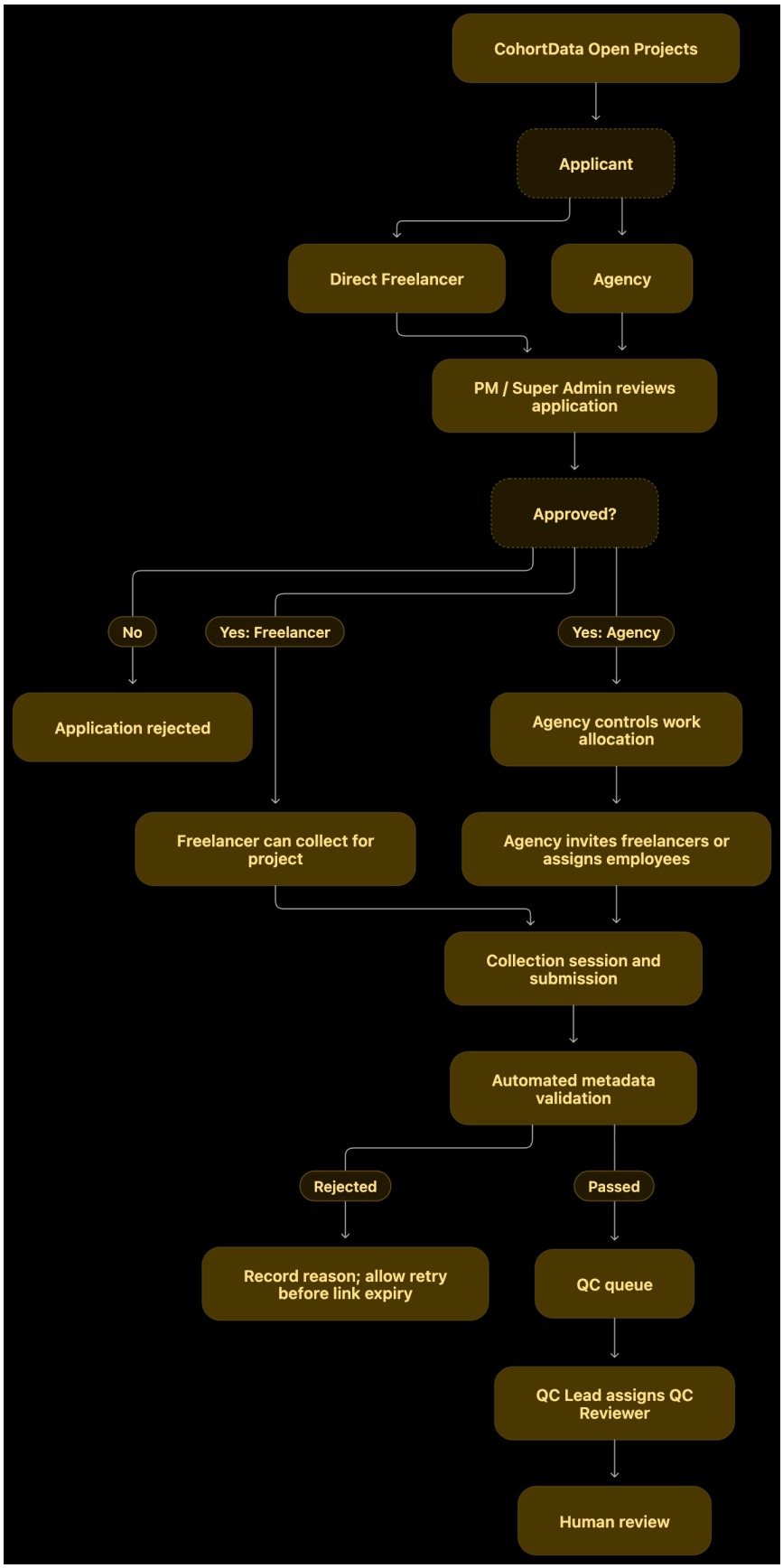
Got it. These answers clarify most of our open decisions. I'll treat them as the current requirements for the CohortData database design, while keeping employee project assignment as a topic to revisit before finalizing that schema.

Updated decisions

Topic	Confirmed decision	Database impact
1. Direct freelancer	Registers directly on CohortData, browses Open Projects, and applies.	No agency organization membership required.
2. Agency members	Once the agency is approved for a project, it controls how it distributes work. No individual CohortData	Separate agency project approval from internal work allocation.

Topic	Confirmed decision	Database impact
	approval is required for every member's submission.	
3. Session and resubmission	Rejected submissions can reuse the same link while it remains valid. The link expires after 24 hours.	Preserve attempts and enforce link expiry.
4. Employee access	Agency members log in. The agency manager may assign members to projects, and assigned members can submit.	Project participation must be tracked separately from organization membership.
5. Automated checks	Invalid metadata or failed required validation can be rejected automatically. Passing submissions proceed to the QC queue.	Store automated check results and rejection reasons separately from human reviews.
6. Finance	Payments and invoices are deferred.	Do not finalize finance tables yet.
7. Media storage	Use the company's 2 TB server for now; finalize storage architecture during the collection phase.	Keep file-storage implementation flexible.

Updated overall flow


CohortData Open Projects
Applicant
Direct Freelancer
Agency
PM / Super Admin reviews application
Approved?
No
Yes: Freelancer
Yes: Agency
Application rejected
Agency controls work allocation
Freelancer can collect for project
Agency invites freelancers or assigns employees
Collection session and submission
Automated metadata validation
Rejected
Passed
Record reason; allow retry before link expiry
QC queue
QC Lead assigns QC Reviewer
Human review

One important distinction: agency approval authorizes the agency to work on the project; it does not automatically approve every submission. Each submission still goes through the required validation and QC workflow.

We'll also keep your instruction about employee assignments: before finalizing that specific database schema, we'll revisit how assignments, visibility, sessions, and link expiry should work rather than assume the design is settled.

Next step: We can now refine the `users`, `organizations`, `organization_memberships`, `roles`, and `permissions` tables with their exact columns, primary keys, foreign keys, constraints, and relationships. Then we can proceed to projects and applications.

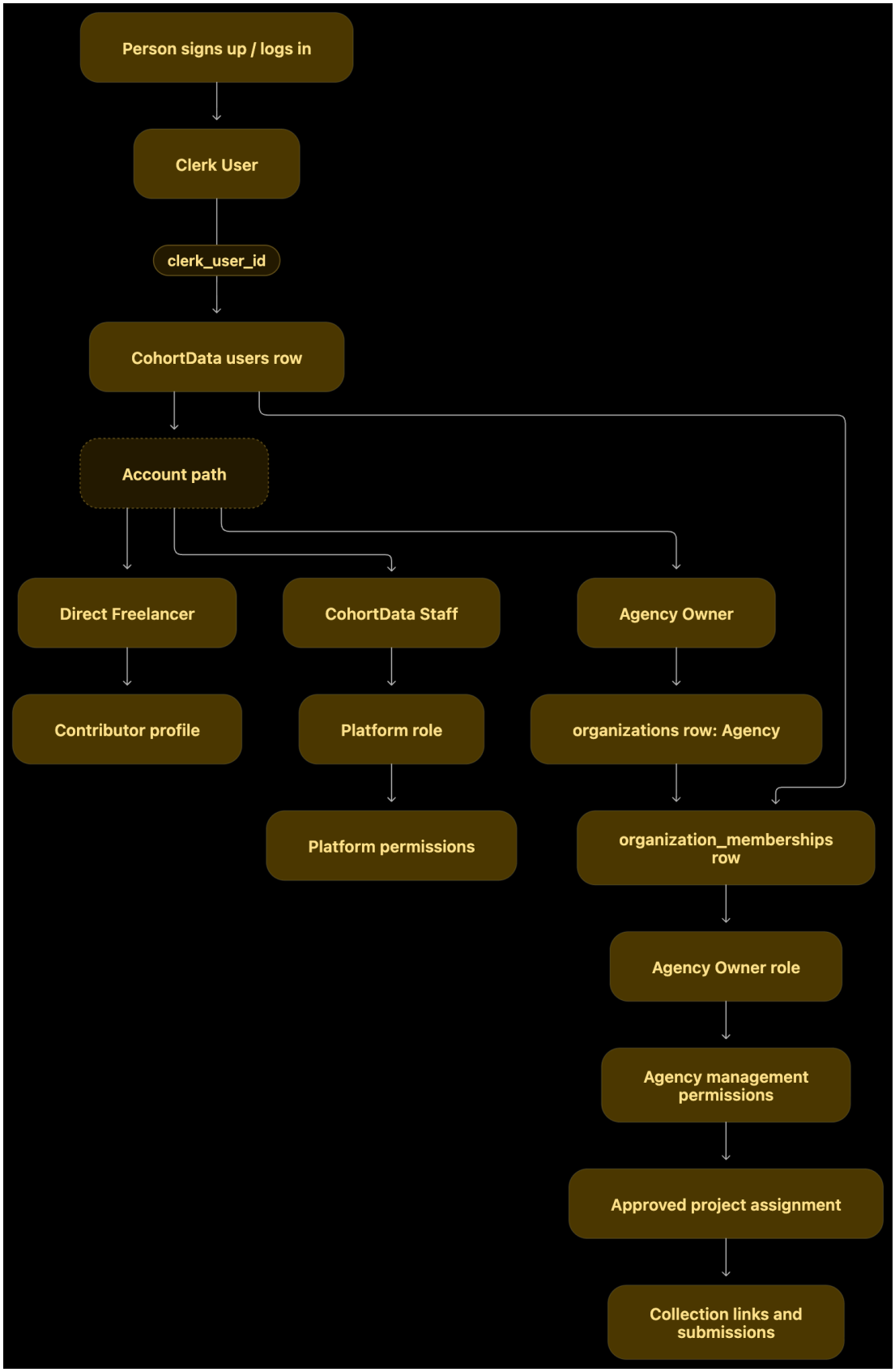
CohortData Database Architecture: Clerk → User → Organization → Roles → Projects

Let's make the database understandable from the ground up. We'll map how a person signing up through Clerk becomes a CohortData user, how they connect to an agency, how an agency owner becomes an admin of that agency, and how permissions flow through the system.

The most important concept is that these are different things:

- Clerk knows who authenticated.
- `users` represents that person in CohortData's database.
- `organizations` represents a company or agency.
- `organization_memberships` connects a person to an organization.
- Roles determine what the person can do.
- Project assignments determine which projects the person or agency is authorized to work on.

1. The complete mapping


Person signs up / logs in
Clerk User
clerk_user_id
CohortData users row
Account path
Direct Freelancer
CohortData Staff
Agency Owner
Contributor profile
Platform role
organizations row: Agency
Platform permissions
organization_memberships row
Agency Owner role
Agency management permissions
Approved project assignment
Collection links and submissions

2. Database terminology, with examples

Imagine the database is a collection of connected spreadsheets, but with rules that prevent invalid relationships.

Term	Meaning	CohortData example
Table	A collection of similar records	`users`
Row / record	One item in a table	One user's database record
Column / field	One property of a record	`email`, `status`
Primary key (PK)	Unique identifier for a record	`users.id`
Foreign key (FK)	A reference to another table's record	`organization_memberships.user_id`
Relationship	How records connect	One agency has many members
One-to-many (1:N)	One record connects to multiple records	One project has many submissions
Unique constraint	Prevents duplicate values or combinations	One membership per user and agency
Index	Helps queries find records faster	Find all submissions for a project
Enum / constraint	Restricts allowed values	Status must be `ACTIVE`, `PENDING`, or `SUSPENDED`
Transaction	A group of database changes that succeed or fail together	Create an agency and its owner membership
RLS	Database rules restricting which rows can be accessed	Agency A cannot read Agency B's private submissions

A foreign key is the glue between tables. Without it, you might have a membership pointing to a user or agency that doesn't exist.

3. The actual tables and how they map together

Let's define the core tables first. These are the proposed PostgreSQL tables for CohortData.

Table A — `users`

Every registered person gets one row in this table, regardless of whether they are an agency owner, freelancer, or CohortData employee.

Column	Type	Meaning
`id`	UUID, PK	CohortData's internal user ID
`clerk_user_id`	TEXT, UNIQUE	The user's ID from Clerk
`email`	TEXT	Email address
`account_status`	TEXT	Active, suspended, etc.
`created_at`	TIMESTAMPTZ	When the record was created

Example:

id	clerk_user_id	email
`usr-101` *	`user_abc123`	owner@agency.com
`usr-102` *	`user_def456`	freelancer@gmail.com

id	clerk_user_id	email
`usr-103` *	`user_ghi789`	admin@cohortdata.com

- Illustrative IDs only. In PostgreSQL, use actual UUID values rather than these shortened examples.

Why have both IDs?

Clerk's ID identifies the authentication account. `users.id` is the stable internal ID that all CohortData business tables reference.

For example, `submissions.contributor_user_id` references `users.id`, not the Clerk ID.

Table B — `organizations`

An organization represents an agency or the parent company.

Column	Type	Meaning
`id`	UUID, PK	Organization ID
`name`	TEXT	Agency/company name
`type`	TEXT	`PLATFORM` or `AGENCY`
`status`	TEXT	Pending verification, active, suspended
`created_at`	TIMESTAMPTZ	Creation time

Example:

id	name	type	status
`org-001` *	CohortData	PLATFORM	ACTIVE
`org-002` *	ABC Data Agency	AGENCY	ACTIVE
`org-003` *	XYZ Data Services	AGENCY	PENDING_VERIFICATION

An agency is not a user. It is a separate business entity that users can belong to.

Table C — `organization_memberships`

This table connects users to organizations.

It solves the question: Which person belongs to which organization?

Column	Type	Meaning
`id`	UUID, PK	Membership ID
`organization_id`	UUID, FK	References `organizations.id`
`user_id`	UUID, FK	References `users.id`
`status`	TEXT	Pending, active, revoked
`joined_at`	TIMESTAMPTZ	When membership became active

Example:

user_id	organization_id	status
`usr-101` *	`org-002` *	ACTIVE

user_id	organization_id	status
`usr-103` *	`org-001` *	ACTIVE

This tells us that the agency owner belongs to ABC Data Agency, while the CohortData employee belongs to the parent organization.

Database relationship: `organizations` 1 → N `organization_memberships`.

One agency can have many members. Each membership row connects one user to one organization.

Because you decided that a user cannot belong to multiple organizations, we would enforce that rule with a unique constraint on `organization_memberships.user_id`, if that remains your final requirement.

4. How does an agency owner become an agency admin?

The user does not become an agency owner just because they signed up with an agency email.

We need an explicit role assignment.

Table D — `roles`

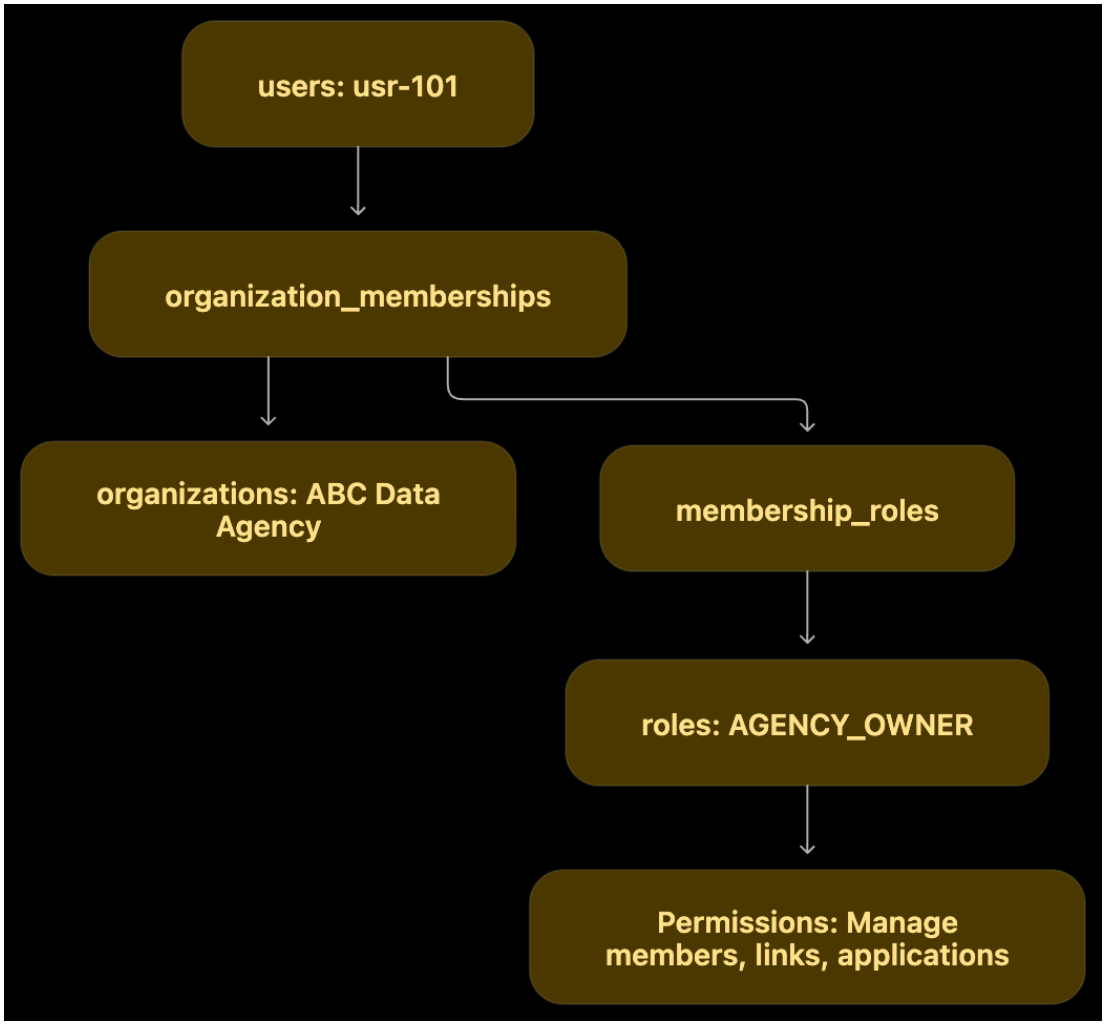
id	code	scope
`role-01` *	`AGENCY_OWNER`	ORGANIZATION
`role-02` *	`AGENCY_MANAGER`	ORGANIZATION
`role-03` *	`PROJECT_MANAGER`	PLATFORM
`role-04` *	`SUPER_ADMIN`	PLATFORM
`role-05` *	`QC_LEAD`	PLATFORM
`role-06` *	`QC_REVIEWER`	PLATFORM
`role-07` *	`FINANCE`	PLATFORM

Table E — `membership_roles`

Column	Type	Meaning
`membership_id`	UUID, FK	Which organization membership
`role_id`	UUID, FK	Which role is assigned

Suppose `usr-101` is the owner of ABC Data Agency.

The relationships look like this:



The owner role is assigned to the membership, not simply to the email address.

That gives us a clean structure:

`users → organization_memberships → membership_roles → roles`

For the parent-team staff, we should use a separate `user_platform_roles` mapping because those roles are platform-wide rather than agency-scoped.

For example:

`users → user_platform_roles → roles: SUPER_ADMIN`

What is a permission?

A role is a named bundle of permissions.

For example:

Permission	Agency Owner	Agency Manager
View agency dashboard	Yes	Yes
View agency submissions	Yes	Yes, if authorized
Invite agency members	Yes	Only if delegated

Permission	Agency Owner	Agency Manager
Change agency ownership	Yes, subject to safeguards	No
View another agency's finances	No	No
Change CohortData platform settings	No	No

We would define these actions in `permissions` and connect them to roles through `role_permissions`.

This is called role-based access control (RBAC).

---

5. How signup works in each scenario

Scenario A — Agency owner signs up

1. User signs up through Clerk.
2. Backend creates the internal `users` row using the Clerk ID.
3. Backend creates an `organizations` row with type `AGENCY`, unless an invited agency already exists.
4. Backend creates the owner's `organization_memberships` row.
5. Backend assigns the `AGENCY_OWNER` role.
6. Agency verification remains pending until CohortData approves it.
7. After verification, the agency can apply for or receive project assignments according to the project workflow.

Scenario B — Direct freelancer signs up

1. User signs up through Clerk.
2. Backend creates a `users` row.
3. Backend creates a `contributor_profiles` row.
4. The freelancer applies for an open project.
5. CohortData approves or rejects the project application.
6. Only an approved freelancer can collect data for that project.

Scenario C — CohortData employee

1. The employee authenticates through Clerk.
2. Their internal `users` record is created or matched.
3. The employee receives a platform role, such as `PROJECT_MANAGER` or `QC_LEAD`.
4. Backend authorization and RLS restrict actions to that role's permissions.

also about this we have new upates - comming in check this :

Yes, this is a major clarification to the user registration and organization model. It affects Domain 1 (Identity & Organizations), Domain 2 (Roles & Permissions), Domain 4 (Assignments & Applications), and Domain 5 (Contributors & Memberships). The other domains remain largely the same, but they depend on this model.

1. The three registration paths

At signup, the user chooses one of three options.

1. Parent Team

CohortData staff accounts receive platform-level roles, such as Super Admin, Project Manager, Finance, QC Lead, or QC Reviewer.

Access should be granted through authorized staff onboarding, not unrestricted public self-registration as an administrator.

2. Direct Freelancer

Anyone registering without a valid agency employee invitation becomes a direct freelancer. They create their account, complete their profile, browse Open Projects, and apply. A Project Manager or Super Admin reviews each application.

No agency organization or agency membership is created for them.

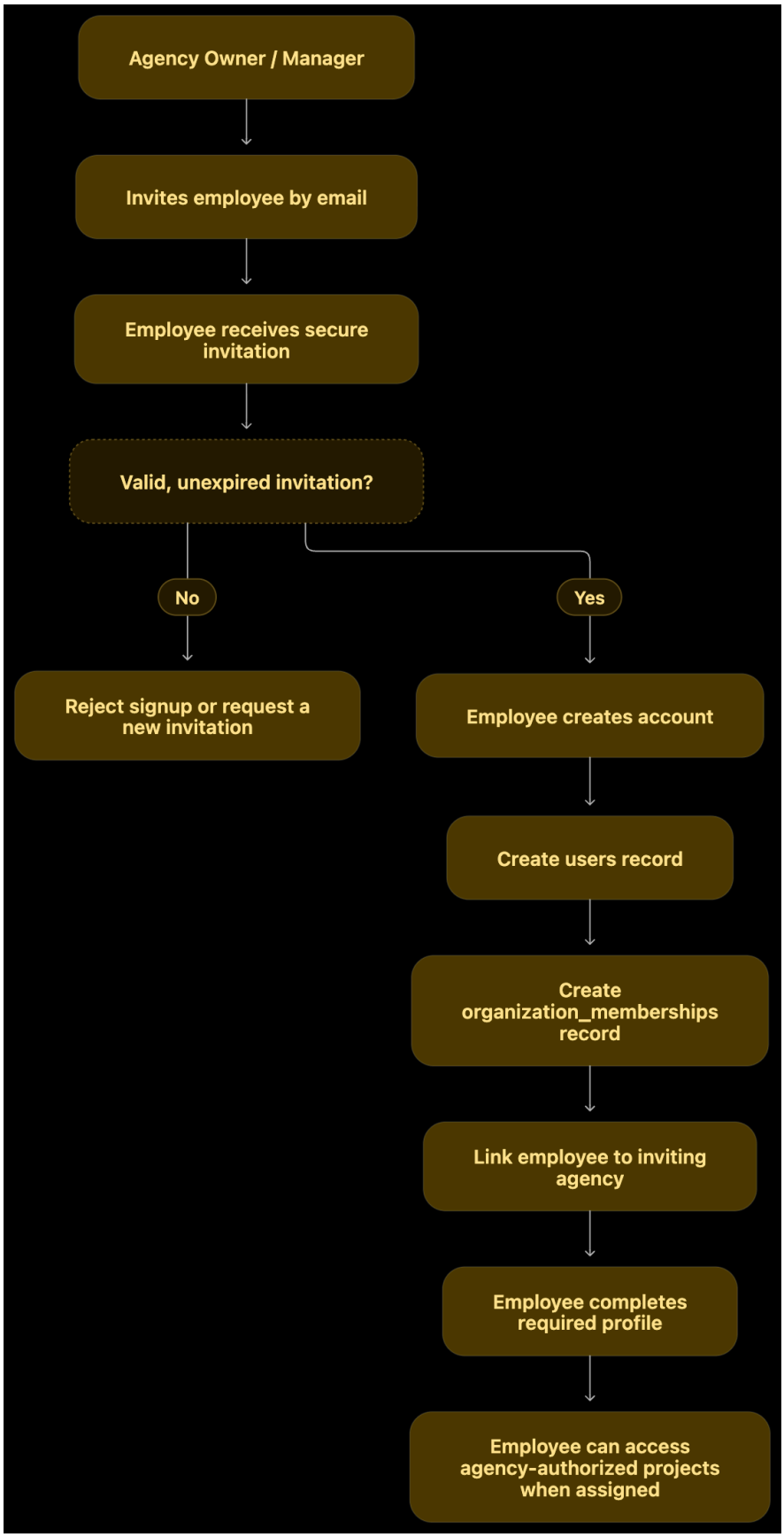
3. Vendor / Agency

A person registers an agency or accepts a direct invitation from CohortData to become an agency owner. The agency must pass CohortData's verification process before it can participate in projects.

Agency ownership and agency verification are separate from project approval.

Agency employees do not get a fourth public registration path. They can register as employees of an agency only after receiving a valid invitation from that agency's authorized manager or owner.

2. Employee invitation flow


Agency Owner / Manager
Invites employee by email
Employee receives secure invitation
Valid, unexpired invitation?
No
Yes
Reject signup or request a new invitation
Employee creates account
Create users record
Create organization_memberships record
Link employee to inviting agency
Employee completes required profile
Employee can access agency-authorized projects when assigned

The critical rule is that the invitation establishes the employee's organizational identity. The employee must not be able to select an arbitrary agency or join one simply by entering its name or email domain.

If someone visits the public signup page without a valid employee invitation, they follow the direct freelancer path instead.

How the database changes with tthis new updates.

Your `organization_memberships` table now has a clearer purpose: it represents an actual relationship between a registered user and an organization.

`users`

Column	Meaning
`id`	Internal user ID (PK)
`clerk_user_id`	Clerk authentication ID (unique)
`email`	User's email
`account_status`	Active, suspended, etc.

`organizations`

Column	Meaning
`id`	Organization ID (PK)
`name`	Organization name
`type`	`PLATFORM` or `AGENCY`
`status`	Pending verification, active, suspended

`organization_memberships`

Column	Meaning
`id`	Membership ID (PK)
`organization_id`	FK → `organizations.id`
`user_id`	FK → `users.id`
`status`	Pending, active, revoked
`joined_at`	When membership became active

`agency_member_invitations`

Column	Meaning
`id`	Invitation ID (PK)
`agency_id`	FK → `organizations.id`
`email`	Invited employee's email
`invited_by`	FK → `users.id`
`token_hash`	Hash of the secure invitation token
`expires_at`	Invitation expiry

Column	Meaning
`status`	Pending, accepted, expired, revoked

The invitation is not itself a membership. The membership is created when the invitation is validly accepted and the employee account is linked.

4. Does this affect the entire architecture?

Domain	Impact
1. Identity & Organizations	Major change: three registration paths and invitation-based employee onboarding
2. Roles & Permissions	Staff roles cannot be self-assigned; agency roles are granted through authorized onboarding
3. Projects & Requirements	No major change
4. Assignments & Applications	Direct freelancers apply individually; agencies apply as organizations
5. Contributors & Memberships	Major change: employees must be invited and linked to their agency
6. Collection & Sessions	Employee submissions can be attributed to both the employee and their agency
7. Automated Checks & QA/QC	No fundamental change
8. Delivery & Results	No fundamental change
9. Earnings & Finance	No immediate change; finance rules remain deferred
10. Audit & Notifications	Must record invitations, acceptance, revocation, and account/role changes

5. One important security rule

Never trust a signup form's selection of “Parent Team” to grant administrator access. Public signup may offer Direct Freelancer and Vendor/Agency registration; Parent Team access should require an authorized invitation or administrative provisioning.

Likewise, an employee's agency membership must come from a valid invitation, not a client-supplied `organization_id`.

Our revised foundation is now:

`Clerk identity → CohortData user → registration path → organization membership (only when applicable) → role → project authorization`

This preserves the distinction between a direct freelancer, an agency owner, an invited agency employee, and a CohortData staff member. We should use this model as the basis for the detailed schema going forward.

does it affect the entire architecture that we try before yes or not

No, it does not change the entire architecture. It changes specific parts of the database design.

- Major changes: Domain 1 (Identity & Organizations) and Domain 5 (Contributors & Memberships).
- Related changes: Domain 2 (Roles & Permissions) and Domain 4 (Assignments & Applications).

- Minor adjustments: Domain 6 (Collection & Sessions) and Domain 10 (Audit & Notifications), mainly for employee attribution and invitation tracking.
- Largely unchanged: Domains 3 (Projects), 7 (QA/QC), 8 (Delivery), and 9 (Finance, which we've deferred).

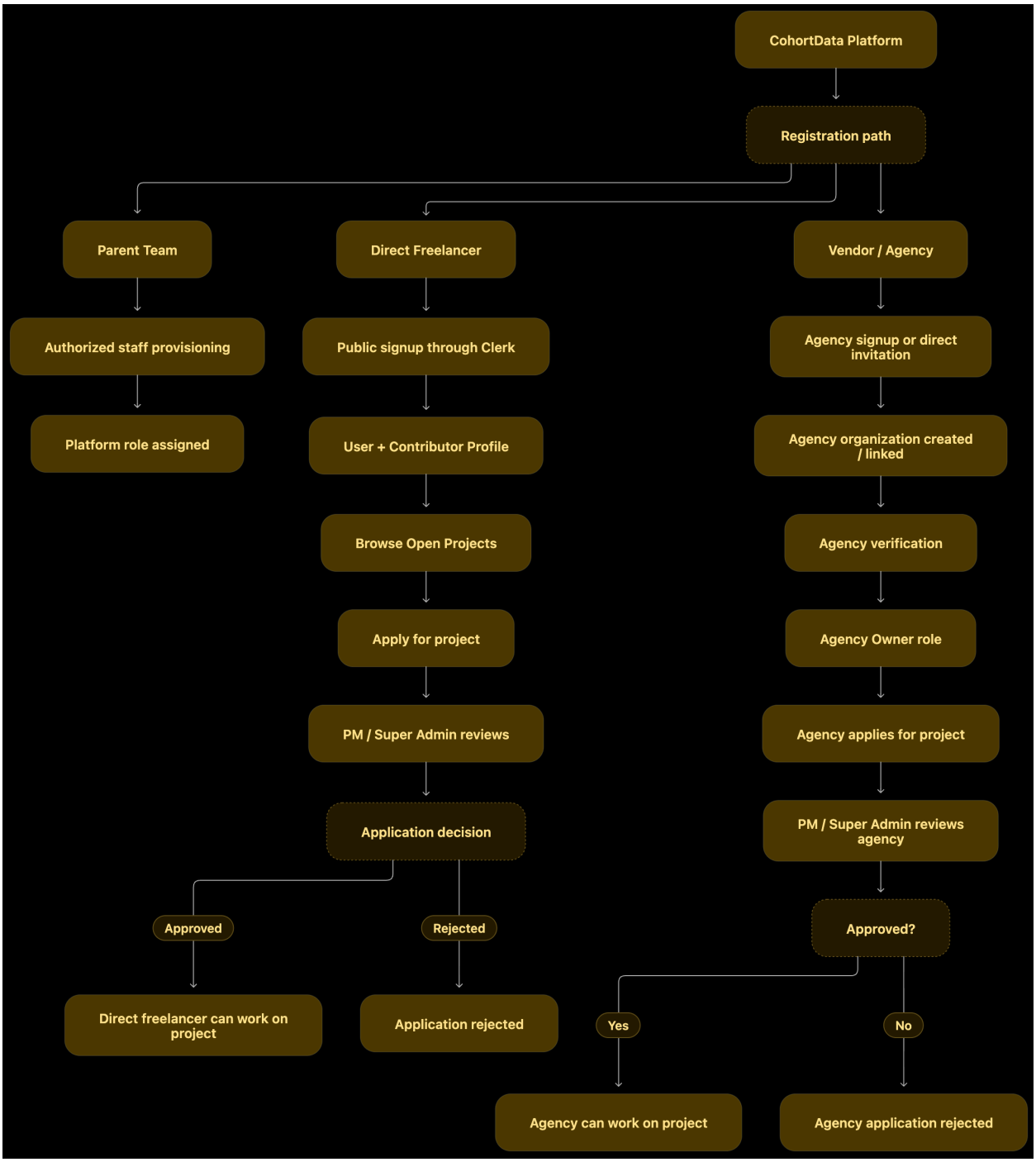
Our overall architecture, technology stack, and core data-collection → QA/QC → delivery workflow remain the same.

Bottom line: We don't need to start over. We need to update the relevant relationships and rules, then continue building the detailed schema.

I'll rebuild the architecture overview with the updated registration rules, including diagrams for the three registration paths, Clerk-to-database mapping, agency invitations, roles, project applications, and the affected domains.

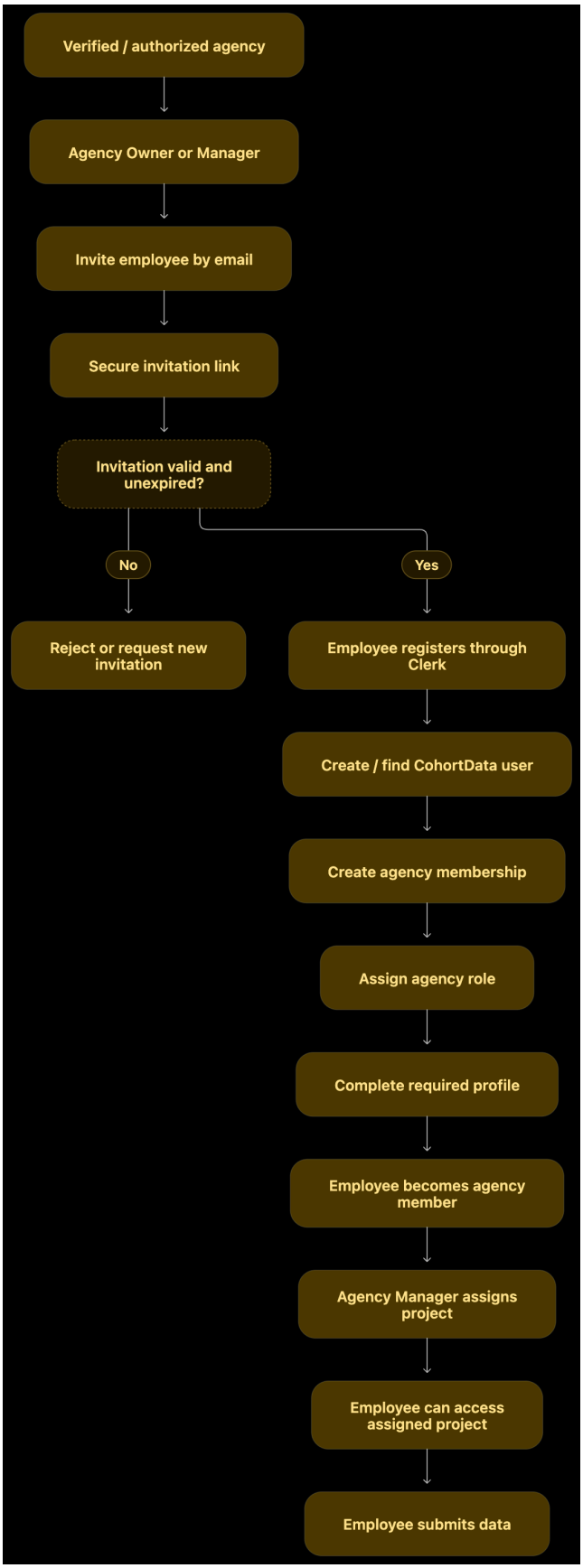
CohortData — Updated Identity & Organization Architecture

1. The three registration paths



Critical rule: Parent Team access cannot be granted merely because a person chooses that option on a public registration form. It requires authorized provisioning or invitation.

2. Agency employee invitation flow

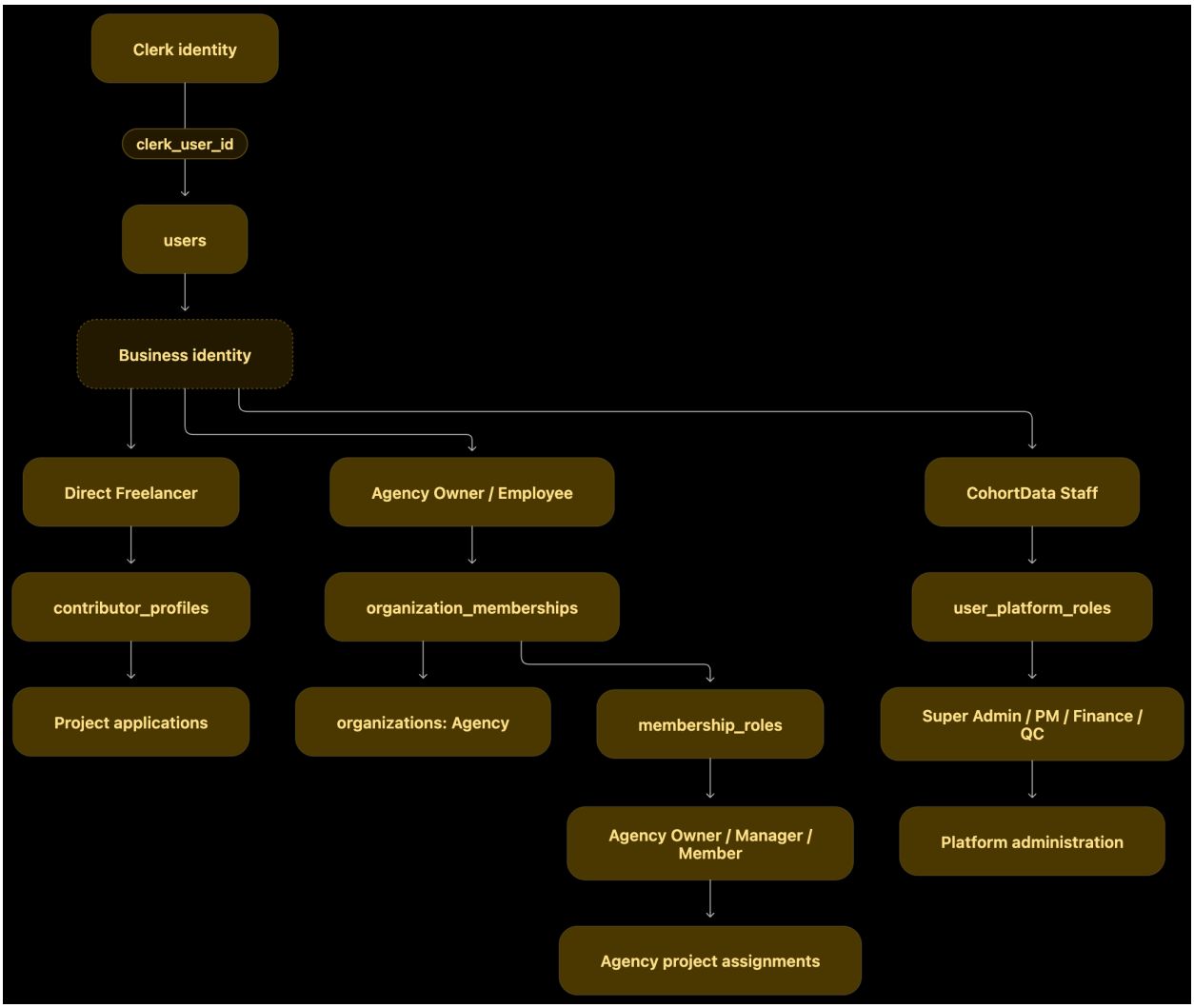

Verified / authorized agency
Agency Owner or Manager
Invite employee by email
Secure invitation link
Invitation valid and unexpired?
No
Yes
Reject or request new invitation
Employee registers through Clerk
Create / find CohortData user
Create agency membership
Assign agency role
Complete required profile
Employee becomes agency member
Agency Manager assigns project
Employee can access assigned project
Employee submits data

An employee cannot join an agency merely by selecting its name or entering its email domain. The invitation is what establishes the authorized relationship.

If someone registers without a valid agency employee invitation, they follow the direct freelancer path.

3. Clerk → User → Organization → Role

This is the central identity mapping for the database.



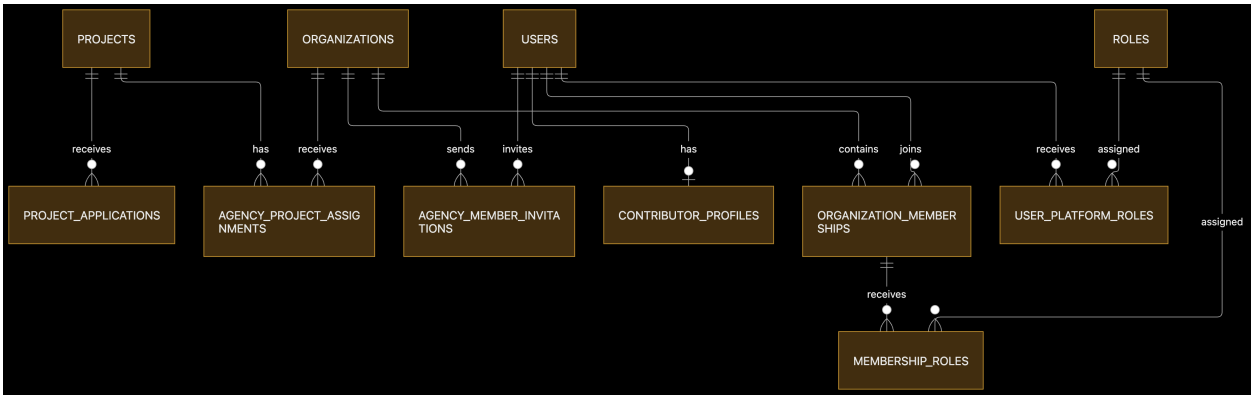
What each table means

Table	Meaning
`users`	One internal record for each registered person
`organizations`	CohortData or an agency
`organization_memberships`	Connects an agency employee or owner to their agency
`contributor_profiles`	Stores contributor-specific profile information
`roles`	Defines available roles

Table	Meaning
`membership_roles`	Assigns roles within an organization
`user_platform_roles`	Assigns CohortData-level roles
`agency_member_invitations`	Controls employee onboarding

A direct freelancer does not need an organization membership. An agency owner or employee does.

4. How the database relationships work



These are conceptual relationships. For example, `USERS ||--o{ ORGANIZATION_MEMBERSHIPS` means a user can have multiple membership records in the general schema, but your current business rule is that each user may belong to at most one organization. We can enforce that with a unique constraint on `organization_memberships.user_id`.

5. Updated domain impact

Domain	What changes
1. Identity & Organizations	Three registration paths; employee membership requires an invitation
2. Roles & Permissions	Staff roles require authorization; agency roles are organization-scoped
3. Projects & Requirements	No fundamental change
4. Assignments & Applications	Agencies apply as organizations; direct freelancers apply individually
5. Contributors & Memberships	Employee onboarding and agency membership must be invitation-based
6. Collection & Sessions	Submissions retain the contributor, project, and agency attribution where applicable
7. Automated Checks & QA/QC	Existing automated and human QC workflow remains
8. Delivery & Results	Existing accepted-data delivery workflow remains
9. Earnings & Finance	Deferred until the business rules are confirmed
10. Audit & Notifications	Track invitations, acceptance, revocation, and role changes

6. Final identity rules

1. Clerk authenticates the user; CohortData stores the business identity.
2. Direct freelancers have no agency membership.

3. Agency employees can join only through a valid invitation from their agency.
4. Agency owners and employees have organization-scoped roles.
5. CohortData staff have platform-scoped roles.
6. Registration does not automatically approve a project application.
7. Agency approval does not require separate CohortData approval for each employee's submission; agency-side project assignment rules still apply.
8. The backend and RLS enforce authorization rather than trusting frontend selections.

This updated architecture replaces the earlier assumption that a freelancer might have an individual organization. The overall project, collection, QC, delivery, and finance domains remain intact, while identity, invitations, membership, and permissions now follow the clarified business rules.